COMP0078 Supervised Learning Cheatsheet

1 Core Linear-Algebra & Convexity Formulas

1.1 Matrix Identities

- $\langle a, b\rangle$: inner product of vectors $a, b \in \mathbb{R}^d$, i.e.

$$\langle a, b\rangle = a^\top b = \sum_{j=1}^{d} a_j b_j$$

- $\|v\|^2 = \langle v, v\rangle = v^\top v$: squared Euclidean norm
- $\|Xw - y\|^2 = (Xw - y)^\top (Xw - y)$: squared loss
- $\mathbf{1}^\top v = \sum_i v_i$: sum of entries of v

1.2 Norm & Inner-Product Inequalities

- **Triangle Inequality:** $\|a + b\| \;\le\; \|a\| + \|b\|$
- **Cauchy–Schwarz:** $|\langle a, b\rangle| \;\le\; \|a\|\,\|b\|$

1.3 Matrix Norms & Trace

- Frobenius norm:

$$\|A\|_F^2 = \sum_{i,j} A_{ij}^2 = \operatorname{trace}(A^\top A)$$

- Trace-cyclicity:

$$\operatorname{trace}(AB) = \operatorname{trace}(BA)$$

- Submultiplicativity:

$$\|AB\| \;\le\; \|A\|\,\|B\|$$

- Operator norm:

$$\|A\|_2 = \sup_{\|x\|=1} \|Ax\|$$

1.4 Convex & Concave Functions

Convexity: $f : \mathbb{R}^d \to \mathbb{R}$ is convex if

$$f\big(\lambda w_1 + (1-\lambda) w_2\big) \;\le\; \lambda f(w_1) + (1-\lambda) f(w_2) \quad \forall\, \lambda \in [0, 1].$$

i.e. The curve of f lies below the line joining any two points (w_1, w_2).

Concavity: f is concave if $-f$ is convex, equivalently

$$f\big(\lambda w_1 + (1-\lambda) w_2\big) \;\ge\; \lambda f(w_1) + (1-\lambda) f(w_2).$$

Jensen's Inequality: for convex f and weights $\lambda_i \ge 0$, $\sum_i \lambda_i = 1$,

$$f\Big(\sum_i \lambda_i x_i\Big) \;\le\; \sum_i \lambda_i\, f(x_i).$$

(used to prove bounds on expected risk, e.g. $f(E[X]) \le E[f(X)]$)

1.5 Hessian-Criterion & First-Order Optimality

1.5.1 Hessian Test

Given f is twice differentiable,

- f convex $\iff$ $\nabla^2 f(w) \succeq 0$ (PSD)
- f strictly convex $\iff$ $\nabla^2 f(w) \succ 0$ (PD) $\implies$ unique minimizer

First-Order Optimality (Convex):

$$\nabla f(w^*) = 0 \implies w^* \text{ is a global minimizer (if convex).}$$

1.6 L–Lipschitz Loss

Let $\ell : \mathbb{R} \times \mathcal{Y} \to \mathbb{R}$ be a loss function. We say that ℓ is L-Lipschitz in its first argument if for every fixed $y \in \mathcal{Y}$, the function $\ell(a, y)$ satisfies:

$$|\ell(a, y) - \ell(b, y)| \le L|a - b| \quad \text{for all } a, b \in \mathbb{R}.$$

Here, $L > 0$ is the **Lipschitz** constant.

2 Least Squares Regression

2.1 Objective Function

Element-wise Form:

$$\min_{w \in \mathbb{R}^d,\ b \in \mathbb{R}} \quad \frac{1}{n} \sum_{i=1}^{n} \left(\langle w, x_i \rangle + b - y_i\right)^2$$

Matrix Form:

$$\min_{w \in \mathbb{R}^d,\ b \in \mathbb{R}} \quad \frac{1}{n} \left\| Xw + b\mathbf{1} - y \right\|^2$$

- $X \in \mathbb{R}^{n \times d}$: design matrix with row vectors $x_i^\top$
- $y \in \mathbb{R}^{n \times 1}$: response vector
- $w \in \mathbb{R}^{d \times 1}$: weight vector
- $b \in \mathbb{R}$: offset (bias) term, not penalized
- $\mathbf{1} \in \mathbb{R}^{n \times 1}$: vector of all ones

2.2 Normal Equation

$$\hat{w} = (X^\top X)^{-1} X^\top y.$$

1. $\hat{w}$: Global Minimizer

- Loss: $L(w) = \|Xw - y\|^2$ is a continuous quadratic.
- Hessian $\nabla^2 L(w) = 2X^\top X$ is PSD L is convex any minimizer is global.
- Hence, optimal $\hat{w}$ always exist, always global.

2. Uniqueness and Rank Conditions

- $X^\top X$ is always symmetric PSD, given $z^\top (X^\top X) z = (Xz)^\top (Xz) = \|Xz\|^2 \ge 0$.
- $X^\top X$ **PD conditions (need to satisfy):**
  - $X^\top X$ is invertible ($\det(X^\top X) \ne 0$)
  - $\operatorname{rank}(X) = d$ (full column rank)
  - Columns of X are linearly independent
- If $X^\top X$ is PD, $\hat{w}$ is the **unique** solution.

3 Ridge Regression

3.1 Objective Function

Element-wise Form:

$$\min_{w \in \mathbb{R}^d,\ b \in \mathbb{R}} \quad \frac{1}{n} \sum_{i=1}^{n} \left(\langle w, x_i \rangle + b - y_i\right)^2 + \lambda \|w\|^2$$

Matrix Form:

$$\min_{w \in \mathbb{R}^d,\ b \in \mathbb{R}} \quad \frac{1}{n} \left\| Xw + b\mathbf{1} - y \right\|^2 + \lambda \|w\|^2$$

- Adds a penalty term $\lambda \|w\|^2 = \lambda w^\top w$ to reduce overfitting
- Bias term b is **not penalized**

3.2 Normal Equation

$$\hat{w}_\lambda = (X^\top X + \lambda I)^{-1} X^\top y$$

1. Strict PD & Invertibility

- $X^\top X + \lambda I$ is always invertible and strictly positive definite.
  - $X^\top X$ is symmetric PSD (proved above) (eigenvalues ≥ 0)
  - Adding λI shifts every eigenvalue by $+\lambda$
  - For any $\lambda > 0$, all eigenvalues > 0, so the matrix is strictly PD.

2. Convexity & Unique $\hat{w}$

- Given $X^\top X + \lambda I \succ 0$, the Hessian of L:
  - $\nabla^2 L_\lambda(w) = 2(X^\top X + \lambda I) \succ 0$
- Given $L_\lambda(w)$ is also continuous and strongly convex, **$\hat{w}$ is a unique global minimizer**.

4 Test Error Metrics

Metric	Formula	Notes
Expected Risk		
$E(f)$	$E(f) \;=\; \mathbb{E}_{(x,y)\sim\rho}\big[\ell\big(f(x), y\big)\big]$	Measures the model's average performance on unseen data drawn from the **true (unknown) distribution** $\rho(x, y)$. In practice, we can't compute it directly as ρ is unknown.
Bayes Estimator		
$f^*(x)$	$f^*(x) \;=\; \arg\min_z \, \mathbb{E}_{y\sim\rho(y\mid x)}\big[\ell(z, y)\big]$	• f^* minimizes the expected risk $E(f^*)$ over all measurable functions. • At each point x, we pick a prediction z that minimizes the expected loss under conditional distribution $\rho(y \mid x)$. • Still **unattainable in practice** because $\rho(y \mid x)$ is unknown too. • For squared loss: $f^*(x) = \mathbb{E}[y \mid x]$
Excess Risk		
	$E(f_S) \;-\; E(f^*)$	Gap between the expected error of learned model (trained on finite dataset S) and optimal Bayes Risk $E(f^*)$ (i.e. how far are we from optimal).

***Derivation of Square Loss Bayes Estimator**
At a fixed input x, define square loss expected risk:

$$g(z) = \mathbb{E}_{y\sim\rho(y|x)}\big[(z-y)^2\big] = \int (z-y)^2 \, \rho(y \mid x) \, dy.$$

We find the minimizer by solving $g'(z) = 0$:

$$
\begin{aligned}
g'(z) &= \frac{d}{dz}\int (z-y)^2\,\rho(y\mid x)\,dy && \text{(Leibniz rule: interchange derivative and integral)}\\
&= \int \frac{d}{dz}(z-y)^2\,\rho(y\mid x)\,dy && \text{(derivative inside)}\\
&= \int 2\,(z-y)\,\rho(y\mid x)\,dy && \text{(chain rule: } d(z-y)^2/dz = 2(z-y)\text{)}\\
&= 2\Big(z\underbrace{\int \rho(y\mid x)\,dy}_{=1} - \underbrace{\int y\,\rho(y\mid x)\,dy}_{=\mathbb{E}[y\mid x]}\Big) && \text{(linearity of integral)}\\
&= 2\big(z-\mathbb{E}[y\mid x]\big).
\end{aligned}
$$

Setting $g'(z)=0$ gives

$$z-\mathbb{E}[y\mid x]=0 \quad\Longrightarrow\quad z=\mathbb{E}[y\mid x].$$

Check second derivative

$$g''(z)=\frac{d}{dz}\big[2(z-\mathbb{E}[y\mid x])\big]=2>0,$$

so this critical point is a minimizer. Hence the Bayes estimator for squared loss is $f^*(x)=\mathbb{E}[y\mid x]$.

4.1 Bias-variance Decomposition

For squared loss:

$$\mathbb{E}[(f_S(x)-f^*(x))^2]=(\mathbb{E}[f_S(x)]-f^*(x))^2+\mathrm{Var}[f_S(x)]$$

- LHS is the **expected excess risk**. (different from the loss, which is between $f_S(x)$ and true label y).
- Bias^2 is how far mean prediction $\mathbb{E}[f_S(x)]$ is from optimal.
- Variance is how learned model fluctuates.

NFL Theorem.

5 PSD Matrix & PD Kernels

1. Core Definitions

- **PSD matrix (linear-algebra level).**
  A *symmetric* matrix $K\in\mathbb{R}^{n\times n}$ is *positive semi-definite* (PSD) if and only if

  $$v^\top K\,v\ \ge 0 \quad \forall\, v\in\mathbb{R}^n.$$

  Gram matrices $X^\top X$ and covariance matrices Σ must be PSD. For Σ we check

  $$v^\top\Sigma v=v^\top\mathbb{E}[(X-\mu)(X-\mu)^\top]v=\mathbb{E}\big[(v^\top(X-\mu))^2\big]\ge 0.$$

- **Gram matrix from a function k.**
  Given $k:\mathbb{X}\times\mathbb{X}\to\mathbb{R}$ and a data set $\{x_1,\dots,x_n\}\subset\mathbb{X}$, the corresponding Gram matrix is

  $$K_{ij}=k(x_i,x_j).$$

  Such a K is **not necessarily symmetric nor PSD** unless k is a valid PD kernel.

2. Local *vs.* Global PSD

- *Local (matrix-level) PSD*: For a ***fixed* data set** the Gram matrix K **may** be PSD. This alone does not guarantee that k comes from an inner product, or a v.
- *Global (function-level) PSD*: A function k is called a *positive-definite kernel* (PD kernel) if the *Gram matrix is PSD* ***for every finite data set***. Formally,

$$K_{ij}=k(x_i,x_j)\quad\Longrightarrow\quad v^\top K\,v\ge 0\quad \forall\, v,\ \forall\{x_1,\dots,x_n\}\subset\mathcal{X}.$$

3. Mercer's Theorem — The Bridge to Inner Products

If k satisfies the *global* PSD condition above, then

- there exists a Hilbert space $\mathcal{H}$, and
- a feature map $\phi : \mathbb{X} \to \mathcal{H}$

such that

$$k(x, x') = \big\langle \phi(x), \phi(x') \big\rangle_{\mathcal{H}} \quad \forall\, x, x' \in \mathbb{X}.$$

If $\mathcal{H}$ is finite-dimensional ($\mathcal{H} \cong \mathbb{R}^D$) the map ϕ can be *explicitly* written; e.g. for polynomial kernels. For kernels like the Gaussian (RBF) kernel, $\mathcal{H}$ is infinite-dimensional but still tractable via the kernel trick.

4. Logical Chain of Implications

$$\boxed{k(x, x') = \langle \phi(x), \phi(x') \rangle} \implies \boxed{K \text{ PSD for } \textit{all} \text{ data sets}} \implies \boxed{k \text{ is a PD kernel}}.$$

Important: The reverse arrow "K PSD for *one* data set $\Rightarrow$ PD kernel" *fails*. Global PSD (all data sets) is required.

5. Practitioner's Checklist

1. To **use** a function k as a kernel, you must know (prove or cite) that it yields a PSD Gram matrix for *every* data set.
2. Once that is true you *never need* the explicit ϕ; simply replace inner products $\langle \phi(x), \phi(x_i) \rangle$ by $k(x, x_i)$ inside your algorithm.
3. One counter-example (a data set whose Gram matrix is *not* PSD) is enough to disqualify k as a kernel.

6 Feature Maps and Kernel

6.1 Least Square in Feature Space

Element-wise Form:

$$\min_{w \in \mathbb{R}^D} \quad \frac{1}{n} \sum_{i=1}^{n} \left(\langle w, \phi(x_i) \rangle - y_i \right)^2$$

Matrix Form:

$$\min_{w \in \mathbb{R}^D} \quad \frac{1}{n} \left\| \Phi w - y \right\|^2$$

- $\phi : \mathbb{R}^d \to \mathbb{R}^D$: feature map transforming x_i into higher-dimensional space D
- $\Phi \in \mathbb{R}^{n \times D}$: feature matrix with row vectors $\phi(x_i)^\top$

6.2 Normal Equation

$$\hat{w} = (\Phi^\top \Phi)^{-1} \Phi^\top y$$

- $\Phi^\top \Phi$: Gram matrix in feature space. Always **symmetric and PSD**.
- A minimizer $\hat{w}$ always exists because the loss is **convex and continuous**
- Since $\Phi^\top \Phi$ is PSD, any solution $\hat{w}$ is a global minimizer
- If $\Phi^\top \Phi$ is invertible (i.e., $\phi(x_i)$ span $\mathbb{R}^D$), the solution $\hat{w}$ is unique

6.3 Computational Complexity: Primal vs Dual

We have two format for same $\hat{w}$ ($\Phi \in \mathbb{R}^{n\times D}$):

- **Primal Form:** $\hat{w} = (\Phi^\top \Phi)^{-1}\Phi^\top y$
- **Dual Form:** $\hat{w} = \Phi^\top \alpha$ where $\alpha = (\Phi\Phi^\top)^{-1} y$
- **Kernelized Dual:** $\hat{w} = \Phi^\top \alpha$ where $\alpha = (K)^{-1} y$ and $K_{ij} = k(x_i, x_j)$
- Polynomial maps of degree p in d dimensions: $D = \binom{d+p}{p}$

Step	Primal (D)	Dual (n)	Kernelized + Dual
Mat. multiplication	$\Phi^\top\Phi \in^{D\times D}$, $O(nD^2)$	$\Phi\Phi^\top \in^{n\times n}$, $O(n^2 D)$	$K_{ij} = k(x_i, x_j) \in^{n\times n}$, $O(n^2 d)$
Mat. inversion	$(\Phi^\top\Phi)^{-1}$, $O(D^3)$	$(\Phi\Phi^\top)^{-1}$, $O(n^3)$	K^{-1}, $O(n^3)$
Final mult.	$\Phi^\top y \in^D$, $O(nD)$	$\Phi^\top \alpha \in^D$, $O(nD)$	$f(x) = \sum_i \alpha_i k(x, x_i)$, $O(nd)$
Time Complexity	$O(D^3 + nD^2)$	$O(n^3 + n^2 D)$	$O(n^3 + n^2 d)$
Memory Complexity	$O(D^2 + nD)$	$O(n^2 + nD)$	$O(n^2)$

[*] Assume the cost to evaluate a single kernel $k(x, x')$ is $O(d)$, where d is **original input dimension** and D is dimension of transformed feature space, so d ¡¡ D almost all the time.

***Derivation of Dual Formation $\hat{w} = \Phi^\top \alpha$**

Primal Objective:

$$\min_{w\in\mathbb{R}^D} \quad \frac{1}{n}\|\Phi w - y\|^2$$

Representer Theorem: The optimal w lies in the span of training feature vector $\Phi^\top$:

$$\hat{w} \in \text{span}(\phi(x_1), \dots, \phi(x_n)) \Rightarrow \exists \alpha \in \mathbb{R}^n \text{ such that } \hat{w} = \Phi^\top \alpha$$

Substitute into loss:

$$L(\alpha) = \frac{1}{n}\|\Phi\Phi^\top \alpha - y\|^2 = \frac{1}{n}\|K\alpha - y\|^2$$
$$\text{where } K = \Phi\Phi^\top \in \mathbb{R}^{n\times n} \text{ is the Gram matrix}$$

Solve for α:

$$\nabla_\alpha L(\alpha) = 0 \quad \Rightarrow \quad K\alpha = y \quad \Rightarrow \quad \alpha = K^{-1} y$$

Obtain dual:

$$\hat{w} = \Phi^\top \alpha = \Phi^\top (\Phi\Phi^\top)^{-1} y$$

The dual approach is preferable when $n < D$, as it avoids **direct inversion of** $\Phi^\top\Phi$.

***Follow-up: Making Predictions via Inner-Products.**

Proved from above the dual solution:

$$\hat{w} = \Phi^\top \alpha, \quad \text{where } \alpha = (\Phi\Phi^\top)^{-1} y$$

Prediction on a new point x is:

$$f(x) = \phi(x)^\top \hat{w} = \phi(x)^\top \Phi^\top \alpha$$

Group terms:

$$f(x) = (\Phi\phi(x))^\top \alpha = \langle \phi(x), \phi(x_i)\rangle \alpha = \sum_{i=1}^n \alpha_i \underbrace{\langle \phi(x), \phi(x_i)\rangle}_{k(x,x_i)} = \sum_{i=1}^n \alpha_i \, k(x, x_i)$$

- $\langle \phi(x), \phi(x_i)\rangle$ is the **inner products** between test point $\phi(x)$ and training feature vectors $\phi(x_i)$. Scalar.
- $\Phi\Phi^\top$ is Gram matrix: $= \langle \phi(x_i), \phi(x_j)\rangle$. i.e. dot products between training points, not a single dot product. $n \times n$ matrix.

We never need to compute $\phi(x)$ or Φ explicitly. All calculations use **kernel evaluations** $k(x, x_i)$ only. Even for computing $\alpha = (\Phi\Phi^\top)^{-1} y$, we only need the **kernel matrix** $K = \Phi\Phi^\top \in \mathbb{R}^{n\times n}$, with:

$$K_{ij} = k(x_i, x_j)$$

6.4 Valid Kernel: How Do We Know?

In writing predictor $f(x) = \sum_{i=1}^{n} \alpha_i \, k(x, x_i)$, we **assumed the existence of a feature map** ϕ such that:

$$k(x, x_i) = \langle \phi(x), \phi(x_i) \rangle$$

In practice, we **never construct** $\phi(x)$, since the feature space D may be very high- or infinite-dimensional. Instead, we want to:

1. Define a function $k(x, x')$ directly (e.g. polynomial),
2. Verify that for any choice of training points x_i , the matrix

$$K_{ij} = k(x_i, x_j) \text{ is symmetric and positive semi-definite (PSD)}$$

3. If so, **Mercer's theorem guarantees** that a feature map ϕ exists and we can replace its dot product with our $k(x_i, x_j)$

***4 Ways to Verify $k(x, x')$ Is a Valid Kernel**
A function $k : \mathcal{X} \times \mathcal{X} \to \mathbb{R}$ is a **valid kernel** (i.e., a *positive definite kernel*) if it satisfies the condition:

$$k(x, x') = \langle \phi(x), \phi(x') \rangle_{\mathcal{H}}$$

for some feature map ϕ into a Hilbert space $\mathcal{H}$, or equivalently, the Gram matrix K is PSD for every dataset.

1. **Explicit $\phi(x)$ Construction** Construct a Hilbert space $\mathcal{H}$ and define a feature map $\phi : \mathcal{X} \to \mathcal{H}$ such that:

$$k(x, x') = \langle \phi(x), \phi(x') \rangle_{\mathcal{H}}$$

   Example: Gaussian kernel with explicit $\phi(x) \in \ell^2$.

2. **Mercer's Theorem** A function k is a valid kernel (i.e., corresponds to some inner product in a feature space) **if and only if**, for any finite dataset $\{x_1, \ldots, x_n\}$, the Gram matrix $K_{ij} = k(x_i, x_j)$ issymmetric and positive semi-definite (PSD):

$$v^\top K v \geq 0 \quad \forall v \in \mathbb{R}^n$$

   which guarantees the existence of an implicit feature map $\phi : \mathcal{X} \to \mathcal{H}$ such that $k(x, x') = \langle \phi(x), \phi(x') \rangle_{\mathcal{H}}$.

3. **Kernel Algebra Rules** Use known closure properties: if k_1, k_2 are valid kernels, then so are:
   - $\alpha k_1(x, x'), \quad$ for $\alpha \geq 0$
   - **Sum:** $k_1(x, x') + k_2(x, x')$
   - **Point-wise product:** $k_1(x, x') \cdot k_2(x, x')$
   - **Polynomial:** $\sum_{m=1}^{p} \alpha_m k(x, x')^m$
   - **Composition:** $k_1(\varphi(x), \varphi(x'))$ for any φ

4. **Counterexample (To Disprove)** To show that k is *not* a valid kernel, find a single dataset $\{x_1, \ldots, x_n\}$ such that the Gram matrix K is not PSD, e.g.:

$$\exists \, v \in \mathbb{R}^n \text{ such that } v^\top K v < 0$$

***Example (1): Verifying Polynomial Kernel via Explicit $\phi(x)$**
First define the polynomial kernel:

$$k(x, x') = (x^\top x' + c)^2, \quad x \in \mathbb{R}^d.$$

$$(x^\top x' + c)^2 = (x^\top x')^2 + 2c(x^\top x') + c^2.$$

Case: $d = 2$ and $x = (x_1, x_2)$

$$(x^\top x')^2 = x_1^2 x_1'^2 + 2x_1 x_2 x_1' x_2' + x_2^2 x_2'^2.$$

We can derive the feature map as **(matching monomials)**:

$$\phi(x) = \left(x_1^2, \sqrt{2}x_1 x_2, x_2^2, \sqrt{2c}x_1, \sqrt{2c}x_2, c\right) \in \mathbb{R}^6.$$

that satisfies

$$\langle \phi(x), \phi(x') \rangle = (x^\top x' + c)^2.$$

Hence, k is positive semi-definite (PSD).

***Example (2): Proving Gaussian Kernel is Valid Kernel**

For inputs $x, z \in \mathbb{R}^d$, the Gaussian kernel is defined as:

$$k(x, z) = \exp\left(-\frac{\|x - z\|^2}{2\sigma^2}\right)$$

1. Factorization

$$k(x, z) = \exp\Big(-\frac{\|x\|^2}{2\sigma^2}\Big) \times \exp\Big(\frac{x^\top z}{\sigma^2}\Big) \times \exp\Big(-\frac{\|z\|^2}{2\sigma^2}\Big).$$

since $\|x - z\|^2 = \|x\|^2 - 2x^\top z + \|z\|^2$.

2. Show each factor is PSD

- $\exp(-\|x\|^2/(2\sigma^2))$. Write

$$k_1(x, z) = \phi_1(x)\, \phi_1(z), \quad \phi_1(x) = \exp\big(-\tfrac{\|x\|^2}{2\sigma^2}\big), \quad \phi_1(z) = \exp\big(-\tfrac{\|z\|^2}{2\sigma^2}\big)$$

  Any kernel of the form $\phi_1(x)\, \phi_1(z)$ is PSD.

- The middle factor can be expanded to sum of exp functions via Taylor Series

$$k_2(x, z) = \exp\big(x^\top z/\sigma^2\big) = \sum_{m=0}^{\infty} \frac{(x^\top z)^m}{m!\, \sigma^{2m}}.$$

  Each monomial kernel $(x^\top z)^m$ is PSD, and nonnegative sums of PSD kernels remain PSD.

3. Product of kernels The product of PSD kernels is PSD. Therefore

$$k(x, z) = k_1(x, z)\, k_2(x, z)\, k_3(x, z)$$

is PSD. Since k is PSD for every finite choice of points, by Mercer's theorem it is a valid positive-definite kernel.

***Classic Kernel Example**

Polynomial Kernel

$$k(x, x') = (x^\top x' + c)^p$$

- $c \in \mathbb{R}_{\geq 0}$: constant (adds bias term)
- $p \in \mathbb{N}$: degree of the polynomial (when $p = 1$, linear kerel)
- Captures global polynomial relationships

Gaussian (RBF) Kernel

$$k(x, x') = \exp\left(-\frac{\|x - x'\|^2}{2\sigma^2}\right)$$

- $\sigma > 0$: kernel width / length scale
- Infinite-dimensional feature space
- Strong non-linearity; smooth similarity measure

7 Hard-Margin SVM

7.1 Optimal Separating Hyperplane

The objective of SVM is to find the **maximum-margin** separating hyperplane:

$$\rho(S) = \max_{w,b} \min_{i \in [m]} \frac{y_i(w^\top x_i + b)}{\|w\|}$$

- A hyperplane is $w^\top x + b = 0$. $\frac{(w^\top x_i + b)}{\|w\|}$ gives signed distance to hyperplane. Multiply by y_i makes it positive if correctly classified, so $\frac{y_i(w^\top x_i + b)}{\|w\|}$ represents margin of (w, b).
- We want the hyperplane that **maximizes the <u>minimum</u> signed margin.**
- This ensures all points are **correctly classified** (signed margin) and the decision boundary is **largest separation between two classes, as far as possible from the closest point**.

7.2 Canonical Parameterization

As the hyperplane defined by parameters (w, b) is *not unique* (scaling by any $\lambda > 0$ yields the same solution), we remove this redundancy by forcing:

$$\min_{i \in [m]} y_i(w^\top x_i + b) = 1.$$

This ensures the margin of the closest point is exactly 1. Then, the objective becomes $\max_{w,b} \rho_S(w, b) = \max_{w,b} \frac{1}{\|w\|} \Leftrightarrow \min_{w,b} \|w\|$.

7.3 Primal Problem (Hard Margin)

This leads to the following convex quadratic problem:

$$\begin{aligned} \min_{w,b} \quad & \frac{1}{2}\|w\|^2 \\ \text{subject to} \quad & y_i(w^\top x_i + b) \geq 1, \quad i = 1, \ldots, m. \end{aligned}$$

- **Minimize** $\frac{1}{2}\|w\|^2$: encourages smaller w and large margin (instead of $\min_{w,b} \|w\|$ for smoother objective).
- **Constraints** $y_i(w^\top x_i + b) \geq 1$ ensure correct classification with **at least margin 1**.
- Convex problem: unique global minimum, only solvable if data is linearly separable.

7.4 Lagrangian and Dual Problem (Hard-Margin SVM)

Introduce Lagrange multipliers $\alpha_i \geq 0$ to encode constraint into objective:

$$L(w, b; \alpha) = \frac{1}{2}\|w\|^2 - \sum_{i=1}^{m} \alpha_i \left(y_i(w^\top x_i + b) - 1 \right)$$

so we turned this into a saddle-point problem: $\min_{w,b} \max_{\alpha \geq 0} L(w, b; \alpha)$.

Set gradients of L to zero for $\hat{w}$ and $\hat{b}$:

$$\frac{\partial L}{\partial w} = 0 \Rightarrow w = \sum_{i=1}^{m} \alpha_i y_i x_i$$

$$\frac{\partial L}{\partial b} = 0 \Rightarrow \sum_{i=1}^{m} \alpha_i y_i = 0$$

Substitute w and b into L to eliminate minimization and w, b:

$$\begin{aligned} \max_{\alpha} \quad & \sum_{i=1}^{m} \alpha_i - \frac{1}{2} \sum_{i,j=1}^{m} \alpha_i \alpha_j y_i y_j x_i^\top x_j \\ \text{subject to} \quad & \sum_{i=1}^{m} \alpha_i y_i = 0, \quad \alpha_i \geq 0 \quad \forall i \end{aligned}$$

Now, purely quadratic in α and kernelizable given the inner product .

***Why convert to Dual?**

1. Dimensional Advantage of Dual

- **Primal** optimizes $w \in \mathbb{R}^d$, where d is the number of features.
- **Dual** optimizes $\alpha \in \mathbb{R}^m$, where m is the number of training samples.
- In kernel methods, primal might involve **infinite** d, but the dual remains **finite** (just m observations). So the dual becomes tractable even for very complex high-dimensional feature maps.

2. The Kernel Trick The dual depends only on dot products $x_i^\top x_j$, so we can replace those with kernels $k(x_i, x_j)$ to avoid computing explicit $\phi(x)$.

***Decomposition of the Dual Problem (for Large m)**

Challenge: Solving dual becomes impractical when m is very large (e.g. $m > 10^5$), since:

- The kernel matrix $K \in \mathbb{R}^{m\times m}$ becomes dense and expensive to store.
- The quadratic programming (QP) solver must optimize over all m variables α_i at once.

Solution: Decomposition (Active Set) Method.

- Maintain a small **active subset** $\mathcal{A} \subseteq \{1, \dots, m\}$ with $|\mathcal{A}| = q \ll m$.
- At each iteration:
  1. Solve the dual problem restricted to α_i for $i \in \mathcal{A}$.
  2. Check which α_i satisfy the **KKT conditions**:

$$\alpha_i \cdot (y_i f(x_i) - 1) = 0, \quad f(x) = \sum_{j=1}^{m} \alpha_j y_j k(x_j, x) + b$$

  3. Remove those that do (as those no need for further update), and add in new variables violating the KKT conditions.
- Repeat until convergence **(When all α_i satisfied KKT, so final α is optimal)**.

7.5 Support Vectors (Hard Margin)

Given the optimal w from dual solution:

$$w = \sum_{i=1}^{m} \alpha_i y_i x_i$$

Only data points with $\alpha_i > 0$ contribute to w, so those x_i where $\alpha_i > 0$ are support vectors! The α vector is typically sparse, so only a small subset of training points are SVs.

7.6 Compute b for Final Solution

From the KKT conditions (Complementary slackness) (Add full KKT conditions in the appendix),

$$\alpha_i \left(y_i (w^\top x_i + b) - 1 \right) = 0$$

- If $\alpha_i > 0$, then $y_i(w^\top x_i + b) = 1$ **(x_i lies exactly on the margin $\Rightarrow$ a support vector)**
- If $y_i(w^\top x_i + b) > 1$, then $\alpha_i = 0$ **(x_i lies strictly outside the margin $\Rightarrow$ not a support vector)**

For any **support vector** x_j with $\alpha_j > 0$ (first case above):

$$b = y_j - w^\top x_j$$

Final classifier: Once we know w, b, and α, we can classify a new x by

$$f(x) = \text{sign}(w^\top b) = \text{sign}\left(\sum_{i=1}^{m} \alpha_i y_i x_i^\top x + b \right)$$

8 Soft Margin SVM

8.1 Primal Problem

Soft margin SVM introduces **slack variables** $\xi_i \geq 0$ to allow constraint violations:

$$\begin{aligned} \min_{w,b,\xi} \quad & \frac{1}{2}\|w\|^2 + C\sum_{i=1}^{m} \xi_i \\ \text{subject to} \quad & y_i(w^\top x_i + b) \geq 1 - \xi_i, \\ & \xi_i \geq 0, \quad i = 1, \dots, m. \end{aligned}$$

- $C\sum \xi_i$: penalizes constraint violations.
  - $\xi_i = 0$: **correctly classified, point lies outside the margin.**
  - $0 < \xi_i < 1$: **inside margin, but still correctly classified.**
  - $\xi_i > 1$: **misclassified.**
- $C > 0$: regularization parameter controlling **the trade-off between margin size and classification error**. The larger C, the narrower margin, more likely **overfit**.

8.2 Primal + Hinge Loss = Unconstrained

Slack-to-Hinge Conversion:

$$\xi_i = 1 - y_i(w^\top x_i + b) = \max(0, 1 - y_i(w^\top x_i + b)) = \max(0, 1 - y\hat{y}) = V_{\text{hinge}}(y, \hat{y})$$

So the constrained objective becomes unconstrained:

$$\min_{w,b} \quad \frac{1}{2}\|w\|^2 + C\sum_{i=1}^{m} \max(0, 1 - y_i(w^\top x_i + b))$$

where the constraints are replaced with a **convex hinge loss** (non-smooth), solvable via standard convex optimization methods.

8.3 Unconstrained Primal = Regularization

The soft-margin SVM primal objective can be interpreted as a regularized ERM problem:

$$\min_{w,b} \quad \underbrace{C\sum_{i=1}^{m} \max(0,\ 1 - y_i(w^\top x_i + b))}_{\text{Empirical Hinge Loss}} + \underbrace{\frac{1}{2}\|w\|^2}_{\text{Regularization Term}}$$

- The first term measures the classification error via the **hinge loss**, which **upper-bounds 0/1 mis-classification loss**.
- The second term penalizes the model complexity by encouraging smaller norm $\|w\|$, i.e., a larger margin.
- This is a special case of the general regularized learning objective:

  $$\min_{w \in \mathbb{R}^d} \quad \sum_{i=1}^{m} \ell(y_i, f_w(x_i)) + \lambda\|w\|^2$$

  where ℓ is the loss function and $\lambda = \frac{1}{2C}$ is the regularization strength.

8.4 Dual Problem: Soft Margin SVM

Given Primal:

$$\min_{w,b,\xi} \quad \frac{1}{2}\|w\|^2 + C\sum_{i=1}^{m} \xi_i$$

$$\text{subject to} \quad y_i(w^\top x_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0$$

We introduce multipliers $\alpha_i, \beta_i \geq 0$:

$$L(w, \xi, b; \alpha, \beta) = \frac{1}{2}\|w\|^2 + C\sum \xi_i - \sum \alpha_i(y_i(w^\top x_i + b) - 1 + \xi_i) - \sum \beta_i \xi_i$$

Solve for the saddle point:

$$w = \sum \alpha_i y_i x_i, \quad \sum \alpha_i y_i = 0, \quad \alpha_i + \beta_i = C \Rightarrow 0 \leq \alpha_i \leq C$$

Dual Optimization Problem:

$$\max_{\alpha} \quad \sum_{i=1}^{m} \alpha_i - \frac{1}{2}\sum_{i,j=1}^{m} \alpha_i \alpha_j y_i y_j x_i^\top x_j$$

$$\text{subject to} \quad \sum_{i=1}^{m} \alpha_i y_i = 0, \quad 0 \leq \alpha_i \leq C$$

The key (only) difference from hard-margin SVM is the **box constraint:** $0 \leq \alpha_i \leq C$ that **limits the contribution of each support vector**. The larger C is, the smaller margin and more likely overfit.

8.5 Support Vectors (Soft Margin)

From KKT Complementary Slackness:

$$\alpha_i \left(y_i(w^\top x_i + b) - 1 + \xi_i\right) = 0, \quad (C - \alpha_i)\xi_i = 0$$

We identify support vectors by:

- $\alpha_i = 0$**:** point is not a support vector (well outside margin).
- $0 < \alpha_i < C$**:** $\delta_i = 0$ **so point lies exactly on the margin, is support vector.**
- $\alpha_i = C$: δ_i may ≥ 0 so point lies **inside the margin** or is misclassified.

Recovering b**:** From any **margin support vector** x_j with $0 < \alpha_j < C$,

$$b = y_j - w^\top x_j$$

Final Decision Rule:

$$f(x) = \text{sign}\left(\sum_{i=1}^{m} \alpha_i y_i x_i^\top x + b\right)$$

Only support vectors with $0 < \alpha_i \leq C$ contribute.

9 SVM - LOO Error Upper Bound

If we train on $m-1$ points and test on the one held out, then repeat for each point, what fraction will be misclassified?

$$\text{LOO Error} \leq \frac{n_{\text{SV}}}{m}$$

- In an SVM, only SVs influence the decision boundary (hyperplane).
- So only when you leave out an SV will the hyperplane shift and might misclassify the test point.
- If there are n_{SV} support vectors, at most all of them get misclassified out of all m points.

10 Kernel + Dual SVM

The dual optimization problem (no matter soft/hard):

$$\begin{aligned} \max_{\alpha} \quad & \sum_{i=1}^{m} \alpha_i - \frac{1}{2} \sum_{i,j=1}^{m} \alpha_i \alpha_j y_i y_j k(x_i, x_j) \\ \text{subject to} \quad & \sum_{i=1}^{m} \alpha_i y_i = 0, \quad 0 \leq \alpha_i \leq C, \quad i = 1, \dots, m \end{aligned}$$

where we replaced dot product $x_i^\top x_j$ with **kernel** $k(x_i, x_j)$**.**

The prediction also updates:

$$f(x) = \sum_{i=1}^{m} \alpha_i y_i \, k(x_i, x) + b$$

and a new data point x is classified via:

$$\hat{y} = \text{sign}(f(x))$$

11 SVM Regression

11.1 ϵ-Insensitive Loss

Loss Function:

$$|y - f(x)|_\epsilon = \max(0, \ |y - f(x)| - \epsilon)$$

- Ignores errors smaller than ϵ (considered acceptable noise).
- Penalizes only deviations that exceed ϵ.

11.2 Objective

Given dataset $\{(x_i, y_i)\}_{i=1}^m$, the Support Vector Regression (SVR) primal problem is:

$$\begin{aligned}
\min_{w,b,\xi,\xi^*} \quad & \frac{1}{2}\|w\|^2 + C\sum_{i=1}^{m}(\xi_i + \xi_i^*) \\
\text{subject to} \quad & y_i - (w^\top x_i + b) \leq \epsilon + \xi_i \\
& (w^\top x_i + b) - y_i \leq \epsilon + \xi_i^* \\
& \xi_i, \xi_i^* \geq 0 \quad \forall i = 1, \dots, m
\end{aligned}$$

- ξ_i, ξ_i^* measure violations above and below the ϵ-tube.
- If $|y_i - f(x_i)| \leq \epsilon$: then $\xi_i = \xi_i^* = 0$.
- Points with **nonzero slack** become support vectors.

12 Convex Surrogates for 0–1 Loss

Loss Name	Formula	Convex?	Smooth?	Comments
0–1 Loss	$\mathbb{I}[r \leq 0]$	×	×	Ideal but intractable
Square Loss	$(1-r)^2$	✓	✓	Upper bound 0-1 Loss.
Hinge Loss	$\max(0, 1-r)$	✓	×	Upper bound 0-1 Loss, non-smooth at $r = 1$
Logistic Loss	$\log(1 + e^{-r})$	✓	✓	Upper bound 0-1 Loss.

- $r = yw^\top x$
- 0–1 Loss isn't convex because it has sharp jump at $r = 0$, not smooth because it's discontinuous at $r = 0$ so not differentiable.
- Hinge, Logistic, and Square loss are **common CONVEX surrogate loss that upper bounds 0–1 loss**:

$$\tilde{\ell}_{\text{surrogate}}(r) \geq \mathbb{I}[r \leq 0]$$

12.1 Logistic Regression

12.1.1 Sigmoid Likelihood

Given observations $S = \{(x_i, y_i)\}_{i=1}^m$ with binary class labels $y_i \in \{-1, 1\}$, and the parameters $w \in \mathbf{R}^\mathbf{d}$, the likelihood is :

$$P(y_i \mid x_i; w) = \frac{1}{1 + e^{-y_i w^\top \phi(x)}}$$

which we want to maximize by choosing $\hat{w}$ that makes observed data most probable.

12.1.2 Optimization Objective

Maximizing $\log P(y_i \mid x_i; w)$ means **minimizing logistic loss** (with regularization):

$$\min_{w \in \mathbb{R}^d} \quad \frac{1}{m}\sum_{i=1}^{m} \log\left(1 + e^{-y_i w^\top x_i}\right) + \lambda\|w\|^2$$

- The first term is the average **logistic loss** over m samples (In ERM we use average loss instead of total loss).
- The second term $\lambda\|w\|^2$ is an ℓ_2 **regularizer**.

12.1.3 Prediction

After we find optimal w, we predict:

$$f(x) = \text{sign}(w^\top x)$$

13 First-Order Optimization

13.1 M-Smooth

A differentiable function $F : \mathbb{R}^d \to \mathbb{R}$ is **M-smooth** if its gradient is **Lipschitz continuous** :

$$\|\nabla F(w) - \nabla F(z)\| \leq M\|w - z\|, \quad \forall w, z \in \mathbb{R}^d$$

- w and z are two arbitrary points in $\mathbb{R}^d$ and $\|w - z\|$ is the Euclidean distance between vectors.
- $M > 0$ is the **Lipschitz constant** of the gradient. (We want the tightest M if possible, but looser is fine.)

***Example: Prove Logistic Loss is M-Smooth**
Consider the logistic loss for $(x, y) \in \mathbb{R}^d \times \{-1, 1\}$:

$$F(w) = \log(1 + e^{-yw^\top x})$$

$$\nabla F(w) = -yx \cdot \sigma(-yw^\top x) \quad \text{where } \sigma(z) = \frac{1}{1 + e^{-z}}$$

Compute gradient difference for any $w, z \in \mathbb{R}^d$:

$$\nabla F(w) - \nabla F(z) = -yx \left[\sigma(-yw^\top x) - \sigma(-yz^\top x)\right]$$

$$\Rightarrow \|\nabla F(w) - \nabla F(z)\| = \|x\| \cdot \left|\sigma(-yw^\top x) - \sigma(-yz^\top x)\right|$$

where $\|a \cdot v\| = |a| \cdot \|v\|$ and $|y| = 1$ as $y \in -1, 1$.
Use the standard sigmoid fact:

$$|\sigma(u) - \sigma(v)| \leq \frac{1}{4}|u - v| \quad \text{for all } u, v \in \mathbb{R}$$

Apply to our case:

$$\left|\sigma(-yw^\top x) - \sigma(-yz^\top x)\right| \leq \frac{1}{4}|y(w - z)^\top x| \leq \frac{1}{4}\|x\| \cdot \|w - z\|$$

where we used Cauchy-Schwarz inequality:

$$|\,(w - z)^\top x\,| \leq \|w - z\| \cdot \|x\|$$

Finally,

$$\|\nabla F(w) - \nabla F(z)\| \leq \frac{1}{4}\|x\|^2 \cdot \|w - z\| \quad \Rightarrow \quad \boxed{M = \frac{1}{4}\|x\|^2}$$

13.2 Quadratic Upper Bound (Descent Lemma)

If F is M-smooth, then around any point w:

$$F(z) \leq F(w) + \nabla F(w)^\top (z - w) + \frac{M}{2}\|z - w\|^2$$

- $F(w) + \nabla F(w)^\top (z - w)$ is a **linear approximation**.
- $\frac{M}{2}\|z - w\|^2$ is a correction term.

***Derivation of Quadratic Upper Bound** Read in detail, still confused.
Assume ∇F is M-Lipschitz:

$$\|\nabla F(u) - \nabla F(v)\| \leq M\|u - v\|, \quad \forall u, v.$$

Define

$$g(t) = F\big(w + t(z - w)\big), \quad t \in [0, 1].$$

By the chain rule,

$$g'(t) = \nabla F\big(w + t(z - w)\big)^\top (z - w).$$

The fundamental theorem of calculus gives

$$F(z) - F(w) = g(1) - g(0) = \int_0^1 g'(t)\, dt.$$

Split the integrand at $t = 0$:

$$F(z) - F(w) = \int_0^1 \nabla F(w)^\top (z-w)\,dt + \int_0^1 \big(\nabla F(w + t(z-w)) - \nabla F(w)\big)^\top (z-w)\,dt$$
$$= \nabla F(w)^\top (z-w) + \int_0^1 \big(\nabla F(w + t(z-w)) - \nabla F(w)\big)^\top (z-w)\,dt.$$

By Cauchy–Schwarz and Lipschitzness,

$$\left|\big(\nabla F(w + t(z-w)) - \nabla F(w)\big)^\top (z-w)\right| \le \|\nabla F(w + t(z-w)) - \nabla F(w)\|\|z-w\| \le Mt\|z-w\|^2.$$

Thus

$$F(z) - F(w) \le \nabla F(w)^\top (z-w) + \int_0^1 Mt\|z-w\|^2\,dt = \nabla F(w)^\top (z-w) + \frac{M}{2}\|z-w\|^2,$$

13.2.1 Use Upper Bound for GD

Let $z = w - \eta \nabla F(w)$. Plug into the bound:

$$F(w - \eta\nabla F(w)) \le F(w) - \eta\left(1 - \frac{M\eta}{2}\right)\|\nabla F(w)\|^2$$

To ensure $F(w)$ is decreasing through this iteration ($\eta\left(1 - \frac{M\eta}{2}\right) \ge 0$),

$$\eta \in \left(0, \frac{2}{M}\right) \quad \text{(safe step sizes)}$$

A common choice is $\eta = \frac{1}{M}$:

$$F(w - \tfrac{1}{M}\nabla F(w)) \le F(w) - \frac{1}{2M}\|\nabla F(w)\|^2$$
$$F(w_{k+1}) \le F(w_t) - \frac{1}{2M}\|\nabla F(w)\|^2$$

which shows how each GD step strictly decreases F by at least $\frac{1}{2M}\|\nabla F(w)\|^2$! Finally, we can define **GD update on** w:

$$w_{k+1} = w_k - \eta\nabla F(w_k)$$

- $\eta > 0$ is the **learning rate** or step size.
- **For vanilla GD on a M-smooth function, $\eta \in \left(0, \frac{2}{M}\right)$ must satisfy.**
- For convex + M-smooth F, GD converges to global minimum w_*.
  - Convex: F has no local minima except the global.
  - M-smooth: the gradient doesn't change too rapidly.

14 Gradient Descent

14.1 Convexity (Lower Bound)

Generally, a function $F : \mathbb{R}^N \to \mathbb{R}$ is **convex** if:

$$F(\theta w + (1-\theta)z) \le \theta F(w) + (1-\theta)F(z), \quad \forall w, z \in \mathbb{R}^N,\ \theta \in [0,1]$$

- All local minima = global.

14.2 First-order Condition

If F is differentiable and **convex** then:

$$F(z) \geq F(w) + \nabla F(w)^\top (z - w), \quad \forall w, z \in \mathbb{R}^N$$

- For a convex F, if $\nabla F(w_*) = 0$, then w_* is a global minimizer:

$$F(z) \geq F(w_*) \quad \forall z \in \mathbb{R}^N$$

***Proof: First-Order Condition Theorem.** (Need to read in detail later!)
(Forward direction $\Rightarrow$): If F is convex and let $\theta \in (0, 1]$. Using the definition of convexity, we obtain:

$$F(\theta z + (1-\theta)w) \leq \theta F(z) + (1-\theta)F(w).$$

Rearranging,

$$\frac{F(\theta z + (1-\theta)w) - F(w)}{\theta} \leq F(z) - F(w).$$

Taking the limit $\theta \to 0$, we obtain:

$$\lim_{\theta \to 0} \frac{F(w + \theta(z - w)) - F(w)}{\theta} = \nabla F(w)^\top (z - w).$$

Thus, we derive:

$$F(z) \geq F(w) + \nabla F(w)^\top (z - w).$$

(Reverse direction $\Leftarrow$): Assume the first-order condition holds and define $u = \theta w + (1-\theta)z$. By assumption, we have:

$$F(w) \geq F(u) + \nabla F(u)^\top (w - u),$$

$$F(z) \geq F(u) + \nabla F(u)^\top (z - u).$$

Multiplying the first equation by θ and the second by $(1-\theta)$, then summing, we obtain:

$$\theta F(w) + (1-\theta)F(z) \geq F(u) = F(\theta w + (1-\theta)z).$$

Thus, F is convex.

14.3 "Sandwich" Bounds: Quadratic UB + Linear LB

Convexity gives a **lower bound (linear)**, while M-smooth gives an **upper bound (quadratic)** of $F(w)$:

$$F(z) \underbrace{\geq}_{\text{convex}} F(w) + \nabla F(w)^\top (z - w) \qquad F(z) \underbrace{\leq}_{\text{smooth}} F(w) + \nabla F(w)^\top (z - w) + \frac{M}{2}\|z - w\|^2$$

If $F(z)$ is convex + differentiable + M-smooth, it satisfies both bounds.

14.4 Convergence of Gradient Descent

Assume that:

- $F : \mathbb{R}^d \to \mathbb{R}$ is **convex and M-smooth** (global minimizer (not unique): $w_* = \arg\min_w F(w)$).
- GD update with fixed step size $\eta = \frac{1}{M}$:

$$w_{k+1} = w_k - \frac{1}{M}\nabla F(w_k)$$

The convergence then has rate $O(1/K)$:

$$F(w_K) - F(w_*) \leq \frac{M}{2K}\|w_0 - w_*\|^2$$

- After K steps, the error is at most $O(1/K)$
- The convergence rate is sublinear (slower than exponential but still guaranteed)

***Proof of the GD Convergence Rate. (IMPORTANT:Need to read in detail later!)** By applying the first-order condition of convexity to w_k and w_*, we obtain:

$$F(w_*) \geq F(w_k) + \nabla F(w_k)^\top (w_* - w_k).$$

$$F(w_k) - F(w_*) \leq \nabla F(w_k)^\top (w_k - w_*).$$

By Quadratic Upper Bound, for $w_{k+1} = w_k - \frac{1}{M}\nabla F(w_k)$,

$$F(w_{k+1}) \leq F(w_k) + \nabla F(w_k)^\top (w_{k+1} - w_k) + \frac{M}{2}\|w_{k+1} - w_k\|^2$$

$$F(w_{k+1}) \leq F(w_k) - \frac{1}{2M}\|\nabla F(w_k)\|^2$$

Next, we write the squared of distance differences:

$$\|w_{k+1} - w_*\|^2 = \|w_k - \frac{1}{M}\nabla F(w_k) - w_*\|^2 = \|w_k - w_*\|^2 - \frac{2}{M}\nabla F(w_k)^\top (w_k - w_*) + \frac{1}{M^2}\|\nabla F(w_k)\|^2$$

Rearrange that for the inner-product term:

$$\nabla F(w_k)^\top (w_k - w_*) = \frac{M}{2}\|w_k - w_*\|^2 - \frac{M}{2}\|w_{k+1} - w_*\|^2 + \frac{1}{2M}\|\nabla F(w_k)\|^2$$

Combining all together,

$$F(w_{k+1}) - F(w_*) \leq \frac{M}{2}\|w_k - w_*\|^2 - \frac{M}{2}\|w_{k+1} - w_*\|^2.$$

Telescope over all iterations $k = 1, \ldots, K$, we obtain:

$$K(F(w_K) - F(w_*)) \leq \frac{M}{2}\|w_0 - w_*\|^2.$$

Dividing by K gives:

$$F(w_K) - F(w_*) \leq \frac{M}{2K}\|w_0 - w_*\|^2.$$

This confirms that gradient descent achieves an $O(1/K)$ convergence rate.

14.5 Estimating Lipschitz Constant

The Lipschitz constant L of function F measures **how fast the function can change**. Let $F : \mathbb{R}^d \to \mathbb{R}$ be differentiable (first order), the optimal M is:

$$L = \sup_w \|\nabla F(w)\|$$

***Derivation of optimal M**

(Forward direction ⇒): If F is Lipschitz continuous, then for any $\theta > 0$ and $v \in \mathbb{R}^d$,

$$|F(w + \theta v) - F(w)| \leq L\|\theta v\| = L\theta\|v\|$$

$$\Rightarrow \frac{|F(w + \theta v) - F(w)|}{\theta} \leq L\|v\|$$

Taking $\theta \to 0$, the LHS becomes **the directional derivative of F at w in the direction v**:

$$\lim_{\theta \to 0} \frac{F(w + \theta v) - F(w)}{\theta} = \nabla F(w)^\top v$$

$$\Rightarrow |\nabla F(w)^\top v| \leq L\|v\|$$

We want a bound for $\|\nabla F(w)\|$. By Cauchy-Schwarz,

$$|\nabla F(w)^\top v| \leq \|\nabla F(w)\|\|v\| \leq L\|v\|$$

$$\Rightarrow \|\nabla F(w)\| \leq L$$

$$\text{so} \quad L \geq \sup_w \|\nabla F(w)\|$$

(Reverse direction $\Leftarrow$): For any two points $w, z \in \mathbb{R}^N$, we use the fundamental theorem of calculus:

$$F(w) - F(z) = \int_0^1 \nabla F(w + \theta(z-w))^\top (z-w) d\theta.$$

Applying the Cauchy-Schwarz inequality:

$$\left| \int_0^1 \nabla F(w + \theta(z-w))^\top (z-w) d\theta \right| \leq \int_0^1 \|\nabla F(w + \theta(z-w))\| d\theta \cdot \|z-w\|.$$

Taking the supremum over all v, we get:

$$\sup_v \|\nabla F(v)\| \geq L.$$

Thus, we conclude $L = \sup_w \|\nabla F(w)\|$.

***Example: Derive Smoothness M for Square Loss**

Given Square Loss:

$$\ell(w) = (w^\top x - y)^2$$

$$\nabla \ell(w) = 2(w^\top x - y)x$$

$$\nabla \ell(w) - \nabla \ell(z) = 2((w-z)^\top x)x$$

$$\|\nabla \ell(w) - \nabla \ell(z)\| = 2|x^\top (w-z)| \cdot \|x\| \leq 2\|x\|^2 \cdot \|w-z\|$$

The square loss is $\boxed{2\|x\|^2}$-smooth (gradient-Lipchitz). Meanwhile $\sup_w \|\nabla \ell(w)\| = \infty$ so it's not function-Lipschitz (no finite L).

15 Sub-gradient Method

For a convex **(not necessarily differentiable, like hinge loss)** function $F : \mathbb{R}^d \to \mathbb{R}$, we replace $\nabla F(w)$ with a sub-gradient $g \in \mathbb{R}^d$ in lower bound:

$$F(z) \geq F(w) + g^\top (z-w), \quad \forall z \in \mathbb{R}^d$$

The set of all sub-gradients g at w is called the **sub-differential**, denoted $\partial F(w)$.

15.0.1 Example: Hinge-Loss Sub-gradient

For $F(w) = \max(0, 1 - yw^\top x)$, the sub-differential is:

$$\partial F(w) = \begin{cases} -yx, & \text{if } yw^\top x < 1 \\ -ryx, \quad r \in [0,1], & \text{if } yw^\top x = 1 \\ 0, & \text{if } yw^\top x > 1 \end{cases}$$

15.1 Sub-gradient Descent Algorithm

We pick any $g_k \in \partial F(w)$:

$$w_{k+1} = w_k - \eta_k g_k, \quad g_k \in \partial F(w_k)$$

15.1.1 Sub-gradient Descent Convergence

Assume F is convex and L-Lipschitz (function). Then:

$$F(w_{\text{best}}) - F(w_*) \leq \frac{\|w_0 - w_*\|^2 + L^2 \sum_{k=1}^K \eta_k^2}{2 \sum_{k=1}^K \eta_k}$$

Choose Constant Step Size: $\eta_k = \eta$

$$F(w_{\text{best}}) - F(w_*) \leq \frac{\|w_0 - w_*\|^2}{2\eta K} + \frac{L^2\eta}{2}$$

Choose $\eta = \frac{1}{\sqrt{K}} \Rightarrow$

$$F(w_{\text{best}}) - F(w_*) \leq \frac{1}{2\sqrt{K}}\left(\|w_0 - w_*\|^2 + L^2\right)$$

Convergence Rate: $\mathcal{O}\left(\frac{1}{\sqrt{K}}\right)$

Note: Sub-gradient converges slower than GD, meaning you need much more iterations for sub-gradient to get close to optimal. **In standard GD, M-Smoothness gives a quadratic upper bound that tightly controls the function's behavior, making gradient steps less wandering, more precise.**

16 Tree-based Models

16.1 CART

CART partitions the input space into disjoint regions $R_1, \ldots, R_P$, and fit constant predictions c_p in each region.

Property	**Regression Tree**	**Classification Tree**
Prediction Output	$\hat{y}(x) \in \mathbb{R}$	$\hat{y}(x) \in \{1, \ldots, K\}$
Model Form	$f(x) = \sum_{p=1}^{P} c_p \mathbb{I}[x \in R_p]$	$f(x) = \arg\max_k \left(\sum_{p=1}^{P} p_{pk} \mathbb{I}[x \in R_p]\right)$
Leaf Node Output	$c_p = \frac{1}{\lvert R_p\rvert} \sum_{x_i \in R_p} y_i$	$p_{pk} = \frac{1}{\lvert R_p\rvert} \sum_{x_i \in R_p} \mathbb{I}[y_i = k]$
Loss Function	Squared Loss: $\ell(y, \hat{y}) = (y - \hat{y})^2$	Misclassification or Impurity-based Loss
Objective Function	$\min_{R_1,\ldots,R_P} \sum_{p=1}^{P} \sum_{x_i \in R_p} (y_i - c_p)^2$	$\min_{R_1,\ldots,R_P} \sum_{p=1}^{P} \lvert R_p\rvert \cdot Q_p(T)$
Node Impurity Measure	Variance within region R_p	$Q_p \in \begin{cases} 1 - \max_k p_{pk} \text{ (Misclassification)} \\ \sum_k p_{pk}(1 - p_{pk}) \text{ (Gini)} \\ -\sum_k p_{pk} \log p_{pk} \text{ (Entropy)} \end{cases}$
Split Optimization	Greedy minimization of within-node squared error	Greedy minimization of impurity (Gini, Entropy, etc.)
Split Form	Binary split $R_1(j, s) = \{x : x_j \leq s\}, R_2(j, s) = \{x : x_j > s\}$	Same as regression
Computational Complexity (1 split)	$O(nm)$	$O(nm)$, same binary split enumeration
Pruning Criterion	Cost-complexity: $C_\lambda(T) = \sum_p m_p Q_p(T) + \lambda\lvert T\rvert$ with $Q_p(T) =$ variance	Same, but $Q_p(T)$ is typically misclassification error

16.1.1 Regression Tree Splitting Criteria

At a candidate split (j, s), we form binary split:

$$R_1(j, s) = \{x : x_j \leq s\}, R_2(j, s) = \{x : x_j > s\}$$

and choose optimal (j, s) via:

$$\Sigma_{x_i \in R_1(j,s)} (y_i - c_1)^2 + \Sigma_{x_i \in R_2(j,s)} (y_i - c_2)^2$$

16.1.2 Classification Tree Splitting Criteria

Let p_{pk} be the proportion of class k in node p, and node is a region for input space containing a subset of training samples. The common impurities are:

- **Mis-classification Error:**

$$Q_p^{\text{error}} = 1 - \max_k p_{pk}$$

  The fraction of points not in the majority class.

- **Gini Index:**
$$Q_p^{\text{gini}} = \sum_k p_{pk}(1 - p_{pk}) = 1 - \sum_k p_{pk}^2$$
Expected error rate if we randomly label a point according to the node distribution.
- **Cross-Entropy (Log Loss):**
$$Q_p^{\text{entropy}} = -\sum_k p_{pk} \log p_{pk}$$
Shannon uncertainty in class labels.
- **Split Selection:** Choose the split that yields the **largest reduction in total impurity** across children:
$$\Delta Q = Q_{\text{parent}} - \left(\frac{m_L}{m_p} Q_L + \frac{m_R}{m_p} Q_R \right)$$
where m_L, m_R are number of points in left/right children, and Q_L, Q_R are impurity of child nodes.

[Node impurity and split gain] Consider a classification tree node with 10 samples: 6 of Class A and 4 of Class B.

$$p_A = 0.6, \quad p_B = 0.4.$$

Compute three impurity measures:
$$\begin{aligned} Q^{\text{error}} &= 1 - \max\{0.6, 0.4\} = 0.4, \\ Q^{\text{Gini}} &= 1 - (0.6^2 + 0.4^2) = 0.48, \\ Q^{\text{Entropy}} &= -\big[0.6 \ln 0.6 + 0.4 \ln 0.4\big] \approx 0.6730. \end{aligned}$$

Now split into left child with counts $(4, 1)$ and right child with counts $(2, 3)$. Their class proportions and impurities are:

Node	Q^{error}	Q^{Gini}	Q^{Entropy}
Left $(4, 1)$	$1 - 0.8 = 0.20$	$1 - (0.8^2 + 0.2^2) = 0.32$	$-[0.8 \ln 0.8 + 0.2 \ln 0.2] \approx 0.500$
Right $(2, 3)$	$1 - 0.6 = 0.40$	$1 - (0.4^2 + 0.6^2) = 0.48$	$-[0.4 \ln 0.4 + 0.6 \ln 0.6] \approx 0.673$

For example, the *Gini gain* from this split is

$$\Delta Q^{\text{Gini}} = Q^{\text{Gini}}_{\text{parent}} - \frac{5}{10} Q^{\text{Gini}}_{\text{left}} - \frac{5}{10} Q^{\text{Gini}}_{\text{right}} = 0.48 - (0.5 \cdot 0.32 + 0.5 \cdot 0.48) = 0.08.$$

16.2 Bagging

16.2.1 Majority Vote Classifier

We have $2T + 1$ independent weak classifiers h_t each with

$$\mathbb{P}[h_t(x) = y] = \frac{1}{2} + \gamma, \gamma > 0$$

The majority vote classifier predicts:

$$H_T(x) := \text{sign}\left(\sum_{t=1}^{2T+1} h_t(x) \right)$$

16.2.2 Exponential-Decay Error Bound

Define the indicator
$$X_t = \mathbf{1}\{h_t(x) = y\}, \quad t = 1, \ldots, 2T + 1,$$
so that $X_t \sim \text{Bernoulli}(p)$ with $p = \frac{1}{2} + \gamma$.

$$X = \sum_{t=1}^{2T+1} X_t \sim \text{Binomial}(2T + 1, p).$$

The ensemble makes an error exactly if $X \leq T$. Mean of X:

$$\mu = \mathbb{X} = (2T + 1)p = T + \tfrac{1}{2} + 2T\gamma.$$

Probability of majority vote classifier is wrong:

$$\Pr\big[H_T(x) \neq y\big] = \Pr\big[X \leq T\big] = \Pr\big[X \leq \mu - \delta\big], \quad \delta = \mu - T = 2T\gamma + \tfrac{1}{2}.$$

By the one-sided Chernoff bound for sums of independent $[0,1]$-variables,

$$\Pr[X \leq \mu - \delta] \;\leq\; \exp\Big(-\frac{\delta^2}{2\mu}\Big).$$

Substituting $\mu = T + \frac{1}{2} + 2T\gamma$ and $\delta = 2T\gamma + \frac{1}{2}$ gives

$$\Pr\big[H_T(x) \neq y\big] \;\leq\; \exp\Big(-\frac{(2T\gamma + \frac{1}{2})^2}{2\,(T + \frac{1}{2} + 2T\gamma)}\Big).$$

For large T this behaves like

$$\Pr[H_T(x) \neq y] \;\lesssim\; \exp\big(-2T\gamma^2\big),$$

so the error decays exponentially in T.

[Note]Chernoff Bound: Let $X_1, \ldots, X_n$ be independent, $X_i \in [0,1]$, with $X = \sum X_i$ and $\mu = \mathbb{E}[X]$. Then for $k \leq \mu$, the Chernoff Bound is:

$$\mathbb{P}(X \leq k) \leq \exp\left(\frac{-(\mu - k)^2}{2\mu}\right)$$

16.2.3 Bootstrap Aggregation (Bagging)

Bagging reduces variance by **train T high-variance base learners on resampled/bootstrapped samples and aggregate via majority vote/average.**

Algorithm 1 Bagging Algorithm

Input: Training data $\mathcal{S} = \{(x_1, y_1), \ldots, (x_m, y_m)\} \subset \mathbb{R}^d \times \{-1, 1\}$
Ensemble Size T
Base learner $h(\cdot)$ (e.g., CART)
Bootstrap sample size M
Output: Final ensemble classifier $H(x)$

for $t = 1$ **to** T **do**
  Sample dataset $\mathcal{S}^{(t)}$ by sampling M times **with replacement** from $\mathcal{S}$ Train base learner $h_t(x) \leftarrow h(\mathcal{S}^{(t)})$
return $H(x) = sign\left(\sum_{t=1}^{T} h_t(x)\right)$

*Majority vote reduces Variance: Var(ensemble) $\approx \frac{\sigma^2}{T}$ (if uncorrelated).

Random Forest is an ensemble of decision trees trained using Bagging (resample subset of observations) + **Random feature selection** at each split when constructing trees. **At each split, only a random subset of features is considered for splitting, which de-correlates the trees and reduces variance.**

16.3 Boosting

Algorithm 2 AdaBoost Algorithm

Input: Training data $\mathcal{S} = \{(x_1, y_1), \ldots, (x_m, y_m)\} \subset \mathbb{R}^d \times \{-1, 1\}$
Ensemble Size T
Base learner $h(\cdot)$
Output: Final classifier: $H(x)$ by aggregating $h_t(x)$ via weighted vote.

for $t = 1$ **to** T **do**
  Initialize: $D_1(i) = \frac{1}{m}$ for all i [Uniform sample weight]
  Train h_t using distribution D_t
  Compute weighted error: $\epsilon_t = \sum_{i=1}^{m} D_t(i) \cdot \mathbb{I}[h_t(x_i) \neq y_i]$
  Compute classifier weight: $\alpha_t = \frac{1}{2} \log\left(\frac{1-\epsilon_t}{\epsilon_t}\right)$ [Larger α_t means trust h_t more!]
  Update data distribution: $D_{t+1}(i) = \frac{D_t(i) \cdot e^{-\alpha_t y_i h_t(x_i)}}{Z_t}$
  where Z_t normalizes D_{t+1} to sum to 1 [Put more weight on samples current ensemble mis-classifies]
return $H(x) = sign\left(\sum_{t=1}^{T} \alpha_t h_t(x)\right)$ *[Combines T weak learners by weighted majority vote.]*

16.3.1 Boosting Convergence & Training Error Bound

If each weak learner satisfies $\epsilon_t \leq \frac{1}{2} - \gamma$ for some $\gamma > 0$, then the **training error** satisfies:

$$\frac{1}{m}\sum_{i=1}^{m} \mathbb{I}[H(x_i) \neq y_i] \leq \exp(-2\gamma^2 T)$$

where the **training error decays exponentially in the number of rounds T.**

***Derivation of Boosting Training Error Bound**

We upper bound 0-1 error using exponential loss:

$$\mathbb{I}[H(x_i) \neq y_i] \leq e^{-y_i f(x_i)} \quad \Rightarrow \quad \frac{1}{m}\sum_{i=1}^{m} \mathbb{I}[H(x_i) \neq y_i] \leq \frac{1}{m}\sum_{i=1}^{m} e^{-y_i f(x_i)}$$

which relaxes the hard loss into a differentiable, smooth surrogate.

Using weight update, we can show:

$$D_{T+1}(i) = \frac{1}{m} \cdot \frac{\prod_{t=1}^{T} e^{-\alpha_t y_i h_t(x_i)}}{\prod_{t=1}^{T} Z_t} \quad \Rightarrow \quad \frac{1}{m}\sum_{i=1}^{m} e^{-y_i f(x_i)} = \prod_{t=1}^{T} Z_t$$

Next, express Z_t Using ϵ_t:

$$Z_t = \sum_i D_t(i) e^{-\alpha_t y_i h_t(x_i)} = e^{-\alpha_t}(1-\epsilon_t) + e^{\alpha_t}\epsilon_t$$

Minimize Z_t w.r.t. α_t:

$$\frac{dZ_t}{d\alpha_t} = 0 \Rightarrow \alpha_t = \frac{1}{2}\log\left(\frac{1-\epsilon_t}{\epsilon_t}\right)$$

Plug in Optimal α_t:

$$Z_t = 2\sqrt{\epsilon_t(1-\epsilon_t)} \quad \text{with} \quad \gamma_t := \frac{1}{2} - \epsilon_t \Rightarrow Z_t = \sqrt{1-4\gamma_t^2}$$

Final Bound on Training Error:

$$\frac{1}{m}\sum_{i=1}^{m} \mathbb{I}[H(x_i) \neq y_i] \leq \prod_{t=1}^{T} Z_t \leq \exp\left(-2\sum_{t=1}^{T}\gamma_t^2\right) \Rightarrow \text{If } \gamma_t \geq \gamma, \text{ then } \boxed{\text{Training Error} \leq e^{-2T\gamma^2}}$$

16.3.2 Boosting as Forward Stagewise Additive Modeling

Boosting can be written as greedily minimizing the exponential-loss risk:

$$\min_{\alpha_1,\ldots,\alpha_T,h_1,\ldots,h_T \in \mathcal{H}} \sum_{i=1}^{m} V\left(y_i, \sum_{t=1}^{T}\alpha_t h_t(x_i)\right)$$

$$V(y,\hat{y}) = \exp(-y\hat{y}).$$

- Upper bounds the 0-1 loss.
- **Is differentiable, convex, smooth.**
- Penalizes misclassified examples heavily (when $y\hat{y} < 0$) and encourages large positive margins.

We build the function f **incrementally**, one basis f at a time:

$$f^{(t-1)} = \sum_{s=1}^{t-1}\alpha_s h_s(x)$$

with updates given by finding optimal α_t and h_t:

$$(\alpha_t, h_t) = \arg\min_{\alpha_t, h_t} \sum_{i=1}^{m} V(y_i, f^{(t-1)}(x_i) + \alpha_t h_t(x_i)).$$

which allows each step focusing on reducing the current loss the most.

***Stage-wise Update Process for AdaBoost**

We seek the best function

$$f_T(x) = \sum_{t=1}^{T} \alpha_t \, h_t(x).$$

by greedily minimizing the empirical exponential loss

$$R(f) = \frac{1}{n} \sum_{i=1}^{n} \exp\bigl(-y_i \, f(x_i)\bigr).$$

At stage t we already have f_{t-1}. We seek $\alpha_t \in \mathcal{R}$ and $h_t \in \mathcal{H}$ that minimise

$$R\bigl(f_{t-1} + \alpha \, h\bigr) = \frac{1}{n} \sum_{i=1}^{n} \exp\bigl(-y_i \, f_{t-1}(x_i)\bigr) \, \exp\bigl(-\alpha \, y_i \, h(x_i)\bigr).$$

1. **Define the sample weights** D_t**.**

$$D_t(i) = \frac{\exp\bigl(-y_i \, f_{t-1}(x_i)\bigr)}{\sum_{j=1}^{n} \exp\bigl(-y_j \, f_{t-1}(x_j)\bigr)}, \qquad \sum_{i=1}^{n} D_t(i) = 1.$$

   With this normalisation the objective (up to a constant factor) becomes

$$\sum_{i=1}^{n} D_t(i) \, \exp\bigl(-\alpha \, y_i \, h(x_i)\bigr).$$

2. **Choose the weak learner** h_t**.**

   For any fixed $\alpha > 0$

$$\exp\bigl(-\alpha \, y_i \, h(x_i)\bigr) = \begin{cases} e^{-\alpha}, & h(x_i) = y_i \text{ (correct)}, \\ e^{+\alpha}, & h(x_i) \neq y_i \text{ (error)}. \end{cases}$$

   Hence the sum is

$$e^{-\alpha}(1 - \hat{\varepsilon}_t(h)) + e^{+\alpha} \hat{\varepsilon}_t(h),$$

   where the *weighted error*

$$\hat{\varepsilon}_t(h) = \sum_{i:\, h(x_i) \neq y_i} D_t(i).$$

   Minimising this for any $\alpha > 0$ is equivalent to minimising $\hat{\varepsilon}_t(h)$ itself, so

$$\boxed{h_t = \arg\min_{h \in \mathcal{H}} \hat{\varepsilon}_t(h)\,.}$$

3. **Choose the coefficient** α_t**.**

   With h_t fixed let $\hat{\varepsilon}_t = \hat{\varepsilon}_t(h_t)$. Minimise $F(\alpha) = e^{-\alpha}(1 - \hat{\varepsilon}_t) + e^{+\alpha} \hat{\varepsilon}_t$ by setting $F'(\alpha) = 0$:

$$\boxed{\alpha_t = \tfrac{1}{2} \ln \frac{1 - \hat{\varepsilon}_t}{\hat{\varepsilon}_t}\,.}$$

4. **Update the weights** D_{t+1}**.**

$$D_{t+1}(i) = \frac{D_t(i) \, \exp\bigl(-\alpha_t \, y_i \, h_t(x_i)\bigr)}{Z_t}, \qquad Z_t = \sum_{j=1}^{n} D_t(j) \, \exp\bigl(-\alpha_t \, y_j \, h_t(x_j)\bigr).$$

   Mis-classified points ($y_i h_t(x_i) = -1$) get weight up multiplied by $e^{+\alpha_t}$; correctly classified points get weight down by $e^{-\alpha_t}$.

5. **Add the new basis to the ensemble.**

$$f_t(x) = f_{t-1}(x) + \alpha_t \, h_t(x).$$

After T stages the final classifier is

$$H(x) = \operatorname{sign}\Bigl(\sum_{t=1}^{T} \alpha_t \, h_t(x)\Bigr).$$

17 Statistical Learning

Let $f : \mathcal{X} \to \mathcal{Y}$ be a predictor and $\rho(x, y)$ the unknown data distribution.

17.1 Fundamentals

17.1.1 Expected Risk (True Risk)

$$E(f) = \int_{\mathcal{X}\times\mathcal{Y}} \ell(f(x), y)\, d\rho(x, y)$$

Measures average loss over the true data distribution $\rho(x, y)$, intractable.

17.1.2 Empirical Risk

$$E_n(f) = \frac{1}{n}\sum_{i=1}^{n} \ell(f(x_i), y_i)$$

- Computed using training data $\{(x_i, y_i)\}_{i=1}^n \sim \rho^n$
- Approximates the expected risk when n is large due to Law of Large Numbers
- **Unbiasedness:** $E_n(f)$ is an unbiased estimator of the expected risk $E(f)$ **(for fixed f independent of the sample only)**:

$$\mathbb{E}_{S\sim\rho^n}[E_n(f)] = \frac{1}{n}\sum_{i=1}^{n} \mathbb{E}_{(x_i,y_i)\sim\rho}[\ell(f(x_i), y_i)] = \frac{1}{n}\sum_{i=1}^{n} E(f) = E(f).$$

- **Convergence:** The expected squared distance converges to 0 as n gets large:

$$\mathbb{E}[(E_n(f) - E(f))^2] = \frac{\mathrm{Var}(\ell(f(x), y))}{n}.$$

  The expected actual distance is bounded by:

$$\mathbb{E}[|E_n(f) - E(f)|] \leq \sqrt{\frac{\mathrm{Var}(\ell(f(x), y))}{n}} = O\left(\frac{1}{\sqrt{n}}\right)$$

  - use Jensen's inequality: $(E[|X|])^2 \leq E[X^2]$
  - this concentration inequality controls **how much empirical risks differ from truth**

17.2 Excess Risk Decomposition

Let f_n be the learned model (e.g. empirical risk minimizer) and $f^* = \arg\min_f E(f)$ be the true risk minimizer. Then:

$$E(f_n) - E(f^*) = (E(f_n) - E_n(f_n)) + (E_n(f_n) - E_n(f^*)) + (E_n(f^*) - E(f^*)).$$

- **Generalization Error** $E(f_n) - E_n(f_n)$: how well f_n generalizes to unseen data. Depends on sample size n, smaller dataset has larger generalization error.
- **Optimization Error** $E_n(f_n) - E_n(f^*)$: how well f_n performs empirically compared to f^*. Depends on the **complexity** of the function space and n.
- **Statistical Noise** $E_n(f^*) - E(f^*)$: sample noise under optimal f^* due to the finite sample containing randomness.
- f_n overfits if $\mathbb{E}[E(f_n) - E(f^*)] > C$ for constant $C > 0$ **[poor generalization]** while $\mathbb{E}[E_n(f_n) - E_n(f^*)] \leq 0$ **[excellent training performance]**.

All three terms vanish as $n \to \infty$, and excess risk $E(f_n) - E(f^*)$ converges to zero — this is the notion of consistency.

17.3 Generalization Error

17.3.1 1. Upper Bound of Expected Excess Risk

Given f_n as **ERM minimizer**, the *expected excess risk* is upper bounded by *generalization bound*:

$$\mathbb{E}[E(f_n) - E(f^*)] \leq \mathbb{E}[E(f_n) - E_n(f_n)].$$

due to the fact that Optimization Error $\mathbb{E}[E_n(f_n) - E_n(f^*)] \leq 0$, and Statistical Noise $\mathbb{E}[E_n(f^*) - E(f^*)] = 0$ (As Unbiasedness of $E_n[f]$), so we get the GE upper bound.

17.3.2 2. Fails Unbiasedness

Unlike fixed f, the ERM minimizer f_n depends on the training data, so:

$$\mathbb{E}[E_n(f_n) - E(f_n)] \neq 0.$$

and the concentration bound doesn't hold either.

17.3.3 3. Finite Hypothesis Spaces: GE Expectation Bound

Let $\mathcal{F}$ be a finite hypothesis space and $f_n \in \mathcal{F}$. Then the **expected Generalization Error** satisfies:

$$\mathbb{E}\,|\mathcal{E}_n(f_n) - \mathcal{E}(f_n)| \leq \mathbb{E}\left[\sup_{f\in\mathcal{F}} |\mathcal{E}_n(f) - \mathcal{E}(f)|\right] \leq \sum_{f\in\mathcal{F}} \mathbb{E}|\mathcal{E}_n(f) - \mathcal{E}(f)| \leq |\mathcal{F}|\sqrt{\frac{V_{\mathcal{F}}}{n}}$$

- First $\leq$: GE can't exceed the **worst-case GE** across the entire $\mathcal{F}$
- The bound shows GE increases with $|\mathcal{F}|$ **(size of the hypothesis space)** and decreases with n.

***Derivation of GE Expectation Bound** For any $f \in \mathcal{F}$ let

$$Z_i^{(f)} := \ell\big(f(x_i), y_i\big), \qquad i = 1, \dots, n,$$

with $Z_i^{(f)} \in [0, 1]$ and variance $V_f := \mathrm{Var}\big(Z_i^{(f)}\big) \leq V_{\mathcal{F}}$.

Step 1 (ERM error $\leq$ supremum). Because the ERM solution f_n is an element of the finite class $\mathcal{F}$,

$$\Big|\mathcal{E}_n(f_n) - \mathcal{E}(f_n)\Big| \;\leq\; \sup_{f\in\mathcal{F}}\Big|\mathcal{E}_n(f) - \mathcal{E}(f)\Big|.$$

Taking expectations gives

$$\mathbb{E}\Big|\mathcal{E}_n(f_n) - \mathcal{E}(f_n)\Big| \;\leq\; \mathbb{E}\Big[\sup_{f\in\mathcal{F}}\big|\mathcal{E}_n(f) - \mathcal{E}(f)\big|\Big]. \tag{A}$$

Note that the supremum over $\mathcal{F}$ no longer uses f_n, so it's independent of the samples. **Step 2 (optional: tail control for each fixed f via Hoeffding).** For any fixed $f \in \mathcal{F}$,

$$\mathbb{P}\Big(\big|\mathcal{E}_n(f) - \mathcal{E}(f)\big| \geq \epsilon\Big) \;\leq\; 2\exp(-2n\epsilon^2) \qquad \text{(Hoeffding)}.$$

Although we will shortly bound *expectations* instead of probability, the Hoeffding bound is kept because it will later yield high-probability guarantees and shows the **same $1/\sqrt{n}$ scaling**.

Step 3 (expected deviation for each f via variance + Jensen). Jensen's inequality ($E[X] \leq \sqrt{E[X^2]}$ applied to the convex map $x \mapsto x^2$ gives

$$\mathbb{E}\big|\mathcal{E}_n(f) - \mathcal{E}(f)\big| \;\leq\; \sqrt{\mathbb{E}\big[\mathcal{E}_n(f) - \mathcal{E}(f)\big]^2} \;=\; \sqrt{\frac{\mathrm{Var}\big(Z_i^{(f)}\big)}{n}} \;\leq\; \sqrt{\frac{V_{\mathcal{F}}}{n}}. \tag{B}$$

- Used the concentration bound above: $\mathbb{E}[|E_n(f) - E(f)|] \leq \sqrt{\frac{\mathrm{Var}(\ell(f(x),y))}{n}}$
- $V_{\mathcal{F}} = \sup_{f\in\mathcal{F}} V_f$ is fixed and is the **max variance of the loss function over $\mathcal{F}$**.
- Note that this bound for GE only works for pre-specified fixed f, so can't use directly but have to plug back to the supreme to recover the $\mathcal{E}_n(f_n) - \mathcal{E}(f_n)$.

Step 4 (supremum $\le$ sum because $\mathcal{F}$ is finite). For non–negative random variables, $\sup_{f\in\mathcal{F}} X_f \le \sum_{f\in\mathcal{F}} X_f$. Applying this and then linearity of expectation,

$$\mathbb{E}\Big[\sup_{f\in\mathcal{F}}\big|\mathcal{E}_n(f)-\mathcal{E}(f)\big|\Big] \;\le\; \sum_{f\in\mathcal{F}} \mathbb{E}\big|\mathcal{E}_n(f)-\mathcal{E}(f)\big| \;\le\; \sum_{f\in\mathcal{F}} \sqrt{\frac{V_{\mathcal{F}}}{n}}.$$

Using the bound in (B) for every f and the fact $\sum_{f\in\mathcal{F}} \sqrt{\frac{V_{\mathcal{F}}}{n}} = |\mathcal{F}|\,\sqrt{\frac{V_{\mathcal{F}}}{n}}$,

$$\mathbb{E}\Big[\sup_{f\in\mathcal{F}}\big|\mathcal{E}_n(f)-\mathcal{E}(f)\big|\Big] \;\le\; |\mathcal{F}|\,\sqrt{\frac{V_{\mathcal{F}}}{n}}. \tag{C}$$

where $|\mathcal{F}|$ denotes the size of the hypothesis space $\mathcal{F}$.

Step 5 (combine (A) and (C)). Insert (C) into (A) to arrive the GE bound for ERM f_n:

$$\boxed{\mathbb{E}\big|\mathcal{E}_n(f_n)-\mathcal{E}(f_n)\big| \;\le\; |\mathcal{F}|\,\sqrt{\frac{V_{\mathcal{F}}}{n}}}.$$

Notes:

- *Role of Hoeffding:* it supplies **exponential tail control** for each fixed f; integrating that tail also recovers a $1/\sqrt{n}$ expectation bound (with a constant $\sqrt{\pi/2}$).
- The result highlights how the generalization gap scales as $O\big(|\mathcal{F}|/\sqrt{n}\big)$ for finite hypothesis classes.
- **This is an expectation bound. A high-probability bound would say** $\mathbb{P}\Big(\big|\mathcal{E}_n(f)-\mathcal{E}(f)\big| \ge \epsilon\Big) \le \delta$

17.3.4 4. Subspaces of Finite Hypotheses

If $\mathcal{H} \subset \mathcal{F}$ is a **finite subspace** of hypothesis (even if $\mathcal{F}$ itself is infinite) and expected risk minimizer $f^* \in \mathcal{H}$. A similar **expected excess risk** bound holds:

$$E[|E(f_n)-E(f^*)|] \le |\mathcal{H}|\sqrt{\frac{V_{\mathcal{H}}}{n}},$$

Restricting the hypothesis space from $\mathcal{F}$ to $\mathcal{H}$ reduces generalization error because $|\mathcal{H}|$ is smaller.

***Derivation of Subspaces Bounds** Read in detail later. Using same derivation, we obtain:

$$\mathbb{E}\Big[\sup_{f\in\mathcal{H}}\big|\mathcal{E}_n(f)-\mathcal{E}(f)\big|\Big] \;\le\; |\mathcal{H}|\,\sqrt{\frac{V_{\mathcal{H}}}{n}}. \tag{C}$$

We know **expected excess risk** is upper bounded by GE:

$$\mathbb{E}|\mathcal{E}(f_n)-\mathcal{E}(f^*)| \le \mathbb{E}\Big|\mathcal{E}_n(f_n)-\mathcal{E}(f_n)\Big| \;\le\; \mathbb{E}\Big[\sup_{f\in\mathcal{F}}\big|\mathcal{E}_n(f)-\mathcal{E}(f)\big|\Big].$$

Hence, we can obtain the **upper bound for expected excess risk**:

$$\mathbb{E}|\mathcal{E}(f_n)-\mathcal{E}(f^*)| \le |\mathcal{H}|\,\sqrt{\frac{V_{\mathcal{H}}}{n}}.$$

17.3.5 5. GE Probability Bound

GE high-probability bound for ERM solution f_n chosen from samples :

$$P\left(|\mathcal{E}_n(f_n)-\mathcal{E}(f_n)| \ge \epsilon\right) \le P\left(\sup_{f\in\mathcal{H}}|\mathcal{E}_n(f)-\mathcal{E}(f)| \ge \epsilon\right) \le 2|\mathcal{H}|\exp\left(-\frac{n\epsilon^2}{2M^2}\right)$$

We guarantee that after searching over $\mathcal{H}$ and get **empirical risk minimizer** f_n, the generalization error depends on $|\mathcal{H}|$, leading to looser bound if $|\mathcal{H}|$is large.

***Derivation of GE Probability Bound via Union + Hoeffding**

Step 1: Apply Hoeffding's Inequality Assume loss function is **bounded**: $|\ell(f(x),y)| \in [0,M]$, then by Hoeffding Inequality, the $\mathcal{E}_n(f)-\mathcal{E}(f)$ has the bound:

$$P\left(|\mathcal{E}_n(f)-\mathcal{E}(f)| \ge \epsilon\right) \le 2\exp\left(-\frac{n\epsilon^2}{2M^2}\right)$$

Larger sample size n or Smaller loss range $M \rightarrow$ tighter bound (better generalization).This is for **a fixed** f, not for the empirical minimizer f_n, which depends on the data!

Step 2: Use Union Bound By Union bound, we can bound the LHS in step 1 with supreme to extend to all $f \in \mathcal{H}$:

$$P\left(\sup_{f\in\mathcal{H}} |\mathcal{E}_n(f) - \mathcal{E}(f)| \geq \epsilon\right) \leq \sum_{f\in\mathcal{H}} P(|\mathcal{E}_n(f) - \mathcal{E}(f)| \geq \epsilon) = \sum_{f\in\mathcal{H}} 2\exp\left(-\frac{n\epsilon^2}{2M^2}\right) = 2|\mathcal{H}|\exp\left(-\frac{n\epsilon^2}{2M^2}\right)$$

Step 3: Link sup back to GE So we can conclude the GE probability bound for all $f \in \mathcal{H}$:

$$P\left(|\mathcal{E}_n(f_n) - \mathcal{E}(f_n)| \geq \epsilon\right) \leq P\left(\sup_{f\in\mathcal{H}} |\mathcal{E}_n(f) - \mathcal{E}(f)| \geq \epsilon\right) \leq 2|\mathcal{H}|\exp\left(-\frac{n\epsilon^2}{2M^2}\right)$$

$$\boxed{P\left(|\mathcal{E}_n(f_n) - \mathcal{E}(f_n)| \geq \epsilon\right) \leq 2|\mathcal{H}|\exp\left(-\frac{n\epsilon^2}{2M^2}\right)}$$

***Definition: Hoeffding's Inequality** Assume that $X_1, \ldots, X_n$ are independent RVs and $X_i \in [a_i, b_i]$. Let $\overline{X} = \frac{1}{n}\sum_{i=1}^n X_i$. Then, for any $\epsilon > 0$:

$$P\left(|\overline{X} - \mathbb{E}[\overline{X}]| \geq \epsilon\right) \leq 2\exp\left(-\frac{2n^2\epsilon^2}{\sum_{i=1}^n (b_i - a_i)^2}\right)$$

i.e. the probability that $\overline{X}$ deviates from its mean by at least ϵ decreases exponentially with n and interval width $b_i - a_i$.

Special Case: When $X_i \in [0,1]$, we have:

$$P\left(|\overline{X} - \mathbb{E}[\overline{X}]| \geq \epsilon\right) \leq 2\exp(-2n\epsilon^2)$$

***Definition: Union Bound**

$$P\left(\bigcup_{i=1}^k A_i\right) \leq \sum_{i=1}^k P(A_i)$$

where $P\left(\bigcup_{i=1}^k A_i\right)$ means the probability that at least one of the events A_i occurs.

17.3.6 6. GE Bound with Confidence Level

Let $\mathcal{H}$ be a finite hypothesis class with a fixed confidence level $\delta \in (0,1)$, the GE for $f_n \in \mathcal{H}$ satisfies with a probability at least $1-\delta$:

$$|\mathcal{E}_n(f_n) - \mathcal{E}(f_n)| \leq \sqrt{\frac{2M^2\log(2|\mathcal{H}|/\delta)}{n}}.$$

where $\mathcal{H}$ is hypothesis class size, M is union bound on the loss $\ell \leq M$, and f_n is ERM solution over $\mathcal{H}$.

***Derivation of GE Bound with δ**

Step 1: Recall GE Probability Bounds

$$\boxed{P\left(|\mathcal{E}_n(f_n) - \mathcal{E}(f_n)| \geq \epsilon\right) \leq 2|\mathcal{H}|\exp\left(-\frac{n\epsilon^2}{2M^2}\right)}$$

We now want to choose ϵ such that RHS equals our desired failure probability δ.

Step 2: Invert the Inequality to Solve for ϵ

We **upper bound** RHS byδ and solve ϵ (to ensure that success probability at least δ):

$$2|\mathcal{H}|\exp\left(-\frac{2n\epsilon^2}{M^2}\right) \leq \delta.$$

$$\log(2|\mathcal{H}|) - \frac{2n\epsilon^2}{M^2} \leq \log\delta.$$

$$\frac{2n\epsilon^2}{M^2} \geq \log\left(\frac{2|\mathcal{H}|}{\delta}\right) \quad \Rightarrow \quad \epsilon^2 \geq \frac{M^2\log(2|\mathcal{H}|/\delta)}{2n}.$$

$$\epsilon \geq \sqrt{\frac{M^2\log(2|\mathcal{H}|/\delta)}{2n}}.$$

Conclusion: With probability at least $1-\delta$,

$$\boxed{|\mathcal{E}_n(f_n) - \mathcal{E}(f_n)| \leq \sqrt{\frac{2M^2\log(2|\mathcal{H}|/\delta)}{n}}}.$$

where $\mathcal{H}_p = 2 \cdot 10^p$.

17.3.7 7. Threshold Functions in GE Bound

Binary classification on $\mathcal{X} = [-1,1]$ with labels in $\{0,1\}$. The true risk minimizer is known as:

$$f^*(x) = \mathbf{1}_{x \geq a^*}, \quad a^* \in [-1,1].$$

We can then discretize thresholds at precision $p > 0$:

$$\mathcal{H}_p \;=\; \Big\{ a \in [-1,1] \;\Big|\; a = \lfloor a \cdot 10^p \rfloor / 10^p \Big\}, \quad |\mathcal{H}_p| = 2 \cdot 10^p.$$

where $|\mathcal{H}_p|$ is the number of possible thresholds.

***Expected Risk Bound with Threshold Function** So the expected-risk bound in the discretized case:

$$\mathbb{E}\big|E(f_n) - E(f^*)\big| \;\leq\; |\mathcal{H}_p| \sqrt{\frac{V_{\mathcal{H}_p}}{n}} \;\leq\; 10^p \sqrt{\frac{V_{\mathcal{H}_p}}{n}}.$$

Remark: Large precision p or small n inflates the generalization bound.

***GE Probability Bound with Threshold Function** With loss $\ell \in [0,1]$, Hoeffding + Union give

$$P\big(|\mathcal{E}_n(f_n) - \mathcal{E}(f_n)| \geq \epsilon\big) \;\leq\; 2\,|\mathcal{H}_p|\,\exp\!\Big(-\tfrac{n\epsilon^2}{2}\Big) = 2 \cdot 10^p\, e^{-n\epsilon^2/2}.$$

Setting the right side to δ and solving for ϵ yields, w.p. $1-\delta$,

$$|\mathcal{E}_n(f_n) - \mathcal{E}(f_n)| \;\leq\; \sqrt{\frac{2\ln(2 \cdot 10^p/\delta)}{n}} \;=\; \sqrt{\frac{2(p \ln 10 + \ln 2 - \ln \delta)}{n}} \;\approx\; \sqrt{\frac{4 + 6p - 2\log(\delta)}{n}}.$$

17.4 Infinite Hypothesis Spaces

$\mathcal{H}_p$ is a finite approx of an infinite hypothesis space by discretizing at precision p. If the true risk minimizer $f^* \notin \mathcal{H}_p$, thenERM on $\mathcal{H}_p$ will never minimize the true risk:

$$\text{Excess Risk: } \mathcal{E}(f_n) - \mathcal{E}(f^*) > 0.$$

As the model complexity $p \to \infty$ increases, $\mathcal{H}_p$ gets closer to infinite space $\mathcal{H}$. However, if p increases too quickly, the GE may be unbounded:

Generalization Error: As model complexity $p \to \infty$, generalization error bound becomes large:

$$|\mathcal{E}_n(f_n) - \mathcal{E}(f_n)| \leq \sqrt{\frac{6p + 18}{n}}.$$

i.e. Overfitting. Too complex model relative to n lead to poor generalization. To control, $p = p(n)$ must grow proportionally with n. Finite $\mathcal{H}_p$ approximates infinite $\mathcal{H}$ well only if p increases proportionally with n.

17.5 Approximation Errors

Now we insert the intermediate "best-in-$\mathcal{H}_p$ model f_p. The excess risk can be decomposed as:

$$E(f_n) - E(f^*) = \underbrace{(E(f_n) - E_n(f_n)) + (E_n(f_n) - E_n(f_p))}_{\text{Generalization Error}} + \underbrace{(E_n(f_p) - E(f_p)) + (E(f_p) - E(f^*))}_{\text{Approximation Error}}.$$

- f^* is the true risk minimizer over everything, possibly even outside infinite $\mathcal{H}$
- f_n is empirical risk minimizer over $\mathcal{H}$ $\surd$
- f_p is the true risk minimizer over $\mathcal{H}$ $\surd$
- **Generalization Error**
  - $(E(f_n) - E_n(f_n))$ is controlled via GE expectation/probability bounds derived above.
  - $(E_n(f_n) - E_n(f_p)) \leq 0$ as f_n is ERM minimizer
- **Approximation Error**:

– $(E_n(f_p) - E(f_p))$ is Generalization Error of f_p. Still probabilistically bounded.

– $(E(f_p) - E(f^*))$ is the irreducible gap as f^* may lie outside $\mathcal{H}_p$ which can't be controlled unless we make assumptions about **the expressiveness of** $\mathcal{H}_{\surd}$

Approximation Error Bound Under regularity assumption that the **marginal distribution** ρ_X has density $r_p(x)$ on $\mathcal{X} = [-1, 1]$:

$$E(f_p) - E(f^*) \leq |a_p - a^*| \cdot \|r_p\|_\infty \cdot 10^{-p}.$$

- $f_p = \mathbb{1}_{x \geq a_p}$ is the closest threshold function to $f^* = \mathbb{1}_{x \geq a^*}$ that exists in the finite space $\mathcal{H}_p$.
- $|a_p - a^*|$ is how far we are from the true threshold a^*.
  – As this term≤ 1 , so error is dominated by $\|r_p\|_\infty \cdot 10^{-p}$.
- **The approx error decreases with increasing** p**, but the rate depend on:**
  – Discretization step 10^{-p} — as $p \to \infty$, f_p approximates f^* more closely.
  – smoothness of the marginal density $r_p(x)$

17.5.1 Generalization vs. Approximation Trade-off

Plugging in the **GE Bound** (derived from "GE Bound with Confidence Level") and **AE Bound** (just derived) to **Excess Risk Decomposition**:

$$E(f_n) - E(f^*) \leq \underbrace{2\sqrt{\frac{4 + 6p - 2\log(\delta)}{n}}}_{2 \text{ Generalization Error}} + \underbrace{\|r_p\|_\infty \cdot 10^{-p}}_{\text{Approximation Error}}.$$

We get the combined upper bound:

$$\phi(n, \delta, p) := 2\sqrt{\frac{4 + 6p - 2\log(\delta)}{n}} + \|r_p\|_\infty \cdot 10^{-p}.$$

as a function of **precision** p **for sample size** n **and confidence level** δ. We then find the best p rhat minimizes the upper bound:

$$p(n, \delta) := \arg\min_{p \geq 0} \phi(n, \delta, p).$$

- Small $n \Rightarrow$ small p precision to prevent overfitting.
- Large $n \Rightarrow$ large p to reduce bias term $\|r_p\|_\infty \cdot 10^{-p}$.

17.6 Regularization

Regularization controls the **complexity of** $\mathcal{H}_\gamma$ as a function of training size n to prevent overfitting. Let the **full infinite hypothesis space** be the union of all candidate hypothesis:

$$\mathcal{H} = \bigcup_{\gamma > 0} \mathcal{H}_\gamma, \quad \text{with } \mathcal{H}_\gamma \subseteq \mathcal{H}_{\gamma'} \text{ for } \gamma \leq \gamma'.$$

where $\gamma > 0$ is the regularization parameter.

17.6.1 Regularization Algorithm

At training time with n samples:

- Choose $\gamma(n)$ so that complexity of $\mathcal{H}_\gamma$ grows as n grows
- Solve ERM over the restricted class $\mathcal{H}_\gamma$:

$$f_{\gamma,n} = \arg\min_{f \in \mathcal{H}_\gamma} \mathcal{E}_n(f).$$

17.6.2 Decomposition of Excess Risk Under Regularization

$$\mathcal{E}(f_{\gamma,n}) - \mathcal{E}(f_*) = \underbrace{\mathcal{E}(f_{\gamma,n}) - \mathcal{E}(f_\gamma)}_{\text{Sample Error (Variance)}} + \underbrace{\mathcal{E}(f_\gamma) - \inf_{f\in\mathcal{H}} \mathcal{E}(f)}_{\text{Approx. Error (Bias)}} + \underbrace{\inf_{f\in\mathcal{H}} \mathcal{E}(f) - \mathcal{E}(f_*)}_{\text{Irreducible Error}}.$$

- f^* is true risk minimizer that **may even lie outside infinite hypothesis space** $\mathcal{H}$ (unless $\mathcal{H}$ is universal).
- f_γ is true risk minimizer in restricted space $\mathcal{H}_\gamma$
- $f_{\gamma,n}$ is the empirical risk minimizer over $\mathcal{H}_\gamma$
- **Sample Error:** how far the empirical minimizer $f_{\gamma,n}$ is from best f_γ. It **decreases as** n **increases**, given enough training data. Always non-negative and expect to be 0 when $n \Rightarrow \infty$.
- **Approximation Error:** how much you lose by restricting to $\mathcal{H}_\gamma$ rather than allowing the full infinite class $\mathcal{H}$. Converges to zero as γ increases as $\mathcal{H}_\gamma$ get very expressive.
  - If $\mathcal{H}$ is a **universal hypothesis space**, then $\lim_{\gamma\to\infty} \mathcal{E}(f_\gamma) - \mathcal{E}(f_*) = 0$ as $\mathcal{H}$ is rich enough to approximate any measurable function, including Bayes optimal f^*.
- **Irreducible Error**: the difference between the **best possible true risk in infinite** $\mathcal{H}$ and the Bayes optimal predictor f^*.
  - This term does not depend on the choice of γ or $\mathcal{H}_\gamma$ because it's a comparison within full class $\mathcal{H}$ versus f^*.
  - If the Bayes optimal predictor f^* is **not in** $\mathcal{H}$, then no matter how much data you have, you will never reach the Bayes risk.

17.6.3 Sample Error Decomposition

It measures how far the empirical minimizer $f_{\gamma,n}$ is from best f_γ within $\mathcal{H}_\gamma$:

$$\mathcal{E}(f_{\gamma,n}) - \mathcal{E}(f_\gamma) = (\mathcal{E}(f_{\gamma,n}) - \mathcal{E}_n(f_{\gamma,n})) + (\mathcal{E}_n(f_{\gamma,n}) - \mathcal{E}_n(f_\gamma)) + (\mathcal{E}_n(f_\gamma) - \mathcal{E}(f_\gamma)),$$

where:

- $\mathcal{E}(f_{\gamma,n}) - \mathcal{E}_n(f_{\gamma,n})$: Generalization error of learned function $f_{\gamma,n}$
- $\mathcal{E}_n(f_\gamma) - \mathcal{E}(f_\gamma)$: Generalization error of f_γ.
  - The GE bound for data-dependent $f_{\gamma,n}$ can be bounded by the expectation bound (or probability):

$$\mathbb{E}\left[\sup_{f\in\mathcal{H}_\gamma} |\mathcal{E}_n(f) - \mathcal{E}(f)|\right] \le |\mathcal{H}_\gamma| \sqrt{\frac{V_{\mathcal{H}_\gamma}}{n}},$$

  - The GE bound for fixed f_γ can be bounded by the concentration bound (or Hoeffding bound):

$$\mathbb{E}[|E_n(f) - E(f)|] \le \sqrt{\frac{\mathrm{Var}(\ell(f(x), y))}{n}}$$

- $\mathcal{E}_n(f_{\gamma,n}) - \mathcal{E}_n(f_\gamma)$: **Empirical excess risk** measuring how far is the learned model from best f_γ (should be ≤ 0 as $f_{\gamma,n}$ is ERM minimizer).

17.7 ERM on Convex Spaces

In infinite $\mathcal{H}$, generalization bounds like $\sup_{f\in\mathcal{H}} |\mathcal{E}_n(f) - \mathcal{E}(f)|$ are hard to control because the GE bounds we derived are based on finite hypothesis. However we can still approx the bound by: (Revisit it later when have more time, lack of details so less important section.)

18 Rademacher Complexity

To derive **tighter** bounds for generalization error, empirical Rademacher complexity:

$$\mathcal{R}_S(\mathcal{H}) = \mathbb{E}_\sigma \left[\sup_{f \in \mathcal{H}} \frac{1}{n} \sum_{i=1}^{n} \sigma_i f(z_i) \right],$$

- **$\mathcal{H}$ can be finite or infinite! Only need to ensure the supremum over $\mathcal{H}$ is finite.**
- $\sigma_i \sim \text{Uniform}(\{-1, +1\})$ are i.i.d. Rademacher random variables.
- $f(z_i)$ could either be **prediction** $f(x_i)$ or **loss values** $l(f(x_i), y_i)$ depending on the context.
- $\sup_{f \in \mathcal{H}}$: The supremum computes the largest average correlation between the functions f in $\mathcal{H}$ and the noise σ_i.
- **$\mathcal{R}_S(\mathcal{H})$ measures the expressive power of hypothesis class $\mathcal{H}$, large $\mathcal{R}_S(\mathcal{H})$ means class too flexible, can align with noise very well, high overfitting risk.**

Given a distribution ρ, the true (expected) Rademacher complexity is:

$$\mathcal{R}_n(\mathcal{H}) = \mathbb{E}_S \left[\mathcal{R}_S(\mathcal{H}) \right], \quad S \sim \rho^n.$$

which **average $\mathcal{R}_S$ over all possible size-n samples drawn from ρ to get the typical complexity of $\mathcal{H}$** .

18.1 GE Bound via Rademacher Complexity

Let $\mathcal{G} = \ell \circ \mathcal{H}$ be the composed loss class. Then:

$$\mathbb{E} \left[\sup_{f \in \mathcal{H}} \mathcal{E}(f) - \mathcal{E}_n(f) \right] \leq 2\mathcal{R}_n(\mathcal{G}) = 2\mathcal{R}_n(\ell \circ \mathcal{H}).$$

i.e. The expected worst-case generalization gap is at most twice the Rademacher complexity of the composed loss class.

***Derivation for GE Bound via RC**

1. **Start with the expected generalization error over sample S:**

$$\mathbb{E}_S \left[\sup_{f \in \mathcal{H}} \mathcal{E}(f) - \mathcal{E}_S(f) \right]$$

2. **Introduce a virtual dataset from same distribution $S' \sim \rho^n$:**

$$\mathcal{E}(f) = \mathbb{E}_{S'}[\mathcal{E}_{S'}(f)] \quad \Rightarrow \quad \mathbb{E}_S \left[\sup_{f \in \mathcal{H}} \mathcal{E}(f) - \mathcal{E}_S(f) \right] = \mathbb{E}_S \left[\sup_{f \in \mathcal{H}} \mathbb{E}_{S'}[\mathcal{E}_{S'}(f) - \mathcal{E}_S(f)] \right]$$

   why $\mathcal{E}_S(f)$ can be pull inside the $\mathbb{E}_{S'}[]$ too? Because it's independent of S', so we can treat it as constant w.r.t. expectation over S'.

3. **Use Jensen's inequality to move expectation outside supremum:** Jensen's inequality states that let X be a RV, and let $\phi : \mathbb{R} \to \mathbb{R}$ be a convex function. Then:

$$\phi(\mathbb{E}[X]) \leq \mathbb{E}[\phi(X)].$$

   Here, $\phi = \sup_{f \in \mathcal{H}}$ is convex over RV $f \mapsto \mathcal{E}_{S'}(f) - \mathcal{E}_S(f)$,

$$\sup \mathbb{E}[\cdot] \leq \mathbb{E}[\sup \cdot],$$

$$\mathbb{E}_S \left[\sup_{f \in \mathcal{H}} \mathbb{E}_{S'}[\mathcal{E}_{S'}(f) - \mathcal{E}_S(f)] \right] \leq \mathbb{E}_{S,S'} \left[\sup_{f \in \mathcal{H}} \mathcal{E}_{S'}(f) - \mathcal{E}_S(f) \right]$$

4. **Express as difference of two empirical risks:**

$$= \mathbb{E}_{S,S'} \left[\sup_{f \in \mathcal{H}} \frac{1}{n} \sum_{i=1}^{n} (\ell(f(x'_i), y'_i) - \ell(f(x_i), y_i)) \right]$$

5. **Introduce Rademacher variables $\sigma_i \in \{-1, 1\}$:** Each σ_i indicates whether we subtract the term from S or S',

$$= \mathbb{E}_{S,S',\sigma} \left[\sup_{f \in \mathcal{H}} \frac{1}{n} \sum_{i=1}^{n} \sigma_i \left(\ell(f(x_i'), y_i') - \ell(f(x_i), y_i) \right) \right]$$

The equality holds because we're essentially averaging over **all possible sign permutations of which dataset comes first**, and because S and S' are drawn **i.i.d. from the same distribution**, flipping samples (x_i, y_i) with (x_i', y_i') doesn't change the distribution.

6. **Apply supremum subadditivity:** We use the property:

$$\sup(A_f - B_f) \leq \sup A_f + \sup B_f$$

$$\leq \mathbb{E}_{S,\sigma} \left[\sup_{f \in \mathcal{H}} \frac{1}{n} \sum_{i=1}^{n} \sigma_i \ell(f(x_i), y_i) \right] + \mathbb{E}_{S',\sigma} \left[\sup_{f \in \mathcal{H}} \frac{1}{n} \sum_{i=1}^{n} \sigma_i \ell(f(x_i'), y_i') \right]$$

7. **Since $S \sim S'$, the two terms are equal:** Since S and S' sample the same number of data points i.i.d from same distribution ρ using same random signs σ, they are statistically indistinguishable. So, their expectations are equal.

$$= 2\, \mathbb{E}_{S,\sigma} \left[\sup_{f \in \mathcal{H}} \frac{1}{n} \sum_{i=1}^{n} \sigma_i \ell(f(x_i), y_i) \right]$$

8. **Define the composed class $\mathcal{G} = \ell \circ \mathcal{H}$:** $\mathcal{G}$ is a sect of loss-evaluated functions. It map each (x, y) to the loss for some $f \in \mathcal{H}$:

$$\mathcal{G} = \{g(x, y) = \ell(f(x), y) : f \in \mathcal{H}\} \quad \Rightarrow \quad \mathcal{R}_n(\mathcal{G}) = \mathbb{E}_{S,\sigma} \left[\sup_{g \in \mathcal{G}} \frac{1}{n} \sum_{i=1}^{n} \sigma_i g(z_i) \right]$$

so the $g(z_i) = \ell(f(x_i), y_i)$.

9. **Final result:**

$$\mathbb{E}_S \left[\sup_{f \in \mathcal{H}} \mathcal{E}(f) - \mathcal{E}_S(f) \right] \leq 2\mathcal{R}_n(\ell \circ \mathcal{H}).$$

19 Contraction Lemma

To reduce GE-RC Bound's dependence on composed class $\mathcal{R}_n(\ell \circ \mathcal{H})$ and express via $\mathcal{R}_n(\mathcal{H})$ instead, we make assumptions on loss ℓ:

19.1 L-Lipschitz Assumption on ℓ

Assume $\ell(f(x), y)$ be an L-Lipschitz function for fixed y, i.e.,

$$|\ell(a, y) - \ell(b, y)| \leq L|a - b| \quad \forall a, b \in \mathbb{R}.$$

meaning that ℓ is not very sensitive to input changes. Then $\mathcal{R}_S(\ell \circ \mathcal{H})$ is upper bounded by:

$$\boxed{\mathcal{R}_S(\ell \circ \mathcal{H}) \leq L \cdot \mathcal{R}_S(\mathcal{H})}$$

so the complexity of the composed class doesn't grow more than L times the original hypothesis class!

***Derivation of Contraction Lemma Upper Bound.** **Step 1: Symmetrize using σ_1** Given the RC of the composed class:

$$\mathcal{R}_S(\ell \circ \mathcal{H}) = \mathbb{E}_\sigma \left[\sup_{f \in \mathcal{H}} \frac{1}{n} \sum_{i=1}^{n} \sigma_i \ell(f(x_i), y_i) \right].$$

We expand the expectation over σ_1 using the **law of total expectation**:

$$\mathcal{R}_S(\ell \circ \mathcal{H}) = \frac{1}{2} \mathbb{E}_\sigma \left[\sup_{f \in \mathcal{H}} \frac{1}{n} \left(\ell(f(x_1), y_1) + \sum_{i=2}^{n} \sigma_i \ell(f(x_i), y_i) \right) \right]$$
$$+ \frac{1}{2} \mathbb{E}_\sigma \left[\sup_{f \in \mathcal{H}} \frac{1}{n} \left(-\ell(f(x_1), y_1) + \sum_{i=2}^{n} \sigma_i \ell(f(x_i), y_i) \right) \right].$$

to explicitly consider σ_1 and split the supremum (split for two conditional expectations given $\sigma_1 = 1$ or -1).

Step 2: Merge suprema using sub-additivity Define two functions $f, f' \in \mathcal{H}$ corresponding to the two sup terms. Then use sub-additivity:

$$\sup_x a + \sup_y b \leq \sup_{x,y}(a+b)$$

$$\mathcal{R}_S(\ell \circ \mathcal{H}) \leq \frac{1}{2}\mathbb{E}_\sigma \left[\sup_{f,f' \in \mathcal{H}} \frac{1}{n} \left(\ell(f(x_1), y_1) - \ell(f'(x_1), y_1) + \sum_{i=2}^{n} \sigma_i [\ell(f(x_i), y_i) + \ell(f'(x_i), y_i)] \right) \right].$$

Step 3: Apply Lipschitz condition Since $\ell(\cdot, y)$ is L-Lipschitz, we have:

$$|\ell(f(x_i), y_i) - \ell(f'(x_i), y_i)| \leq L|f(x_i) - f'(x_i)|.$$

So for the first term in here:

$$|\ell(f(x_1), y_1) - \ell(f'(x_1), y_1)| \leq L|f(x_1) - f'(x_1)|.$$

So, the Rademacher complexity is bounded by:

$$\mathcal{R}_S(\ell \circ \mathcal{H}) \leq \frac{L}{2n} \sup_{f,f'} |f(x_1) - f'(x_1)| + \frac{1}{2}\mathbb{E}_\sigma \left[\sup_{f,f'} \frac{1}{n} \sum_{i=2}^{n} \sigma_i (\ell(f(x_i), y_i) + \ell(f'(x_i), y_i)) \right].$$

Step 4: Recursively apply the same argument and combine terms Repeat the same decomposition on $i = 2$ to n. Each term contributes at most L times the Rademacher complexity of $\mathcal{H}$.

Summing over all n terms gives **(ℓ is gone)**:

$$\mathcal{R}_S(\ell \circ \mathcal{H}) \leq L \cdot \mathbb{E}_\sigma \left[\sup_{f \in \mathcal{H}} \frac{1}{n} \sum_{i=1}^{n} \sigma_i f(x_i) \right] \leq L \cdot \mathcal{R}_S(\mathcal{H}).$$

20 GE Probability Bounds in Infinite $\mathcal{H}$

20.1 McDiarmid's Inequality

Let $\mathcal{Z}$ be a dataset, and let $g : \mathcal{Z}^n \to \mathbb{R}$ be a function such that there exists $c > 0$ satisfying:

$$|g(z_1, \ldots, z_i, \ldots, z_n) - g(z_1, \ldots, z_i', \ldots, z_n)| \leq c,$$

then, let $Z_1, \ldots, Z_n$ be i.i.d. over distribution $\mathcal{Z}$. Then, for any $\delta > 0$, with probability at least $1 - \delta$, we have:

$$|g(Z_1, \ldots, Z_n) - \mathbb{E}[g(Z_1, \ldots, Z_n)]| \leq c\sqrt{\frac{n}{2} \log \frac{2}{\delta}}.$$

meaning if g doesn't change much by altering one sample input, then g also concentrates tightly around its mean. **This upper bound quantifies how unlikely it is for g to deviate too far from its mean.** Slightly similar to Hoeffding's inequality, which is a specific case where $g(Z)$ is sum of independent bounded RVs: $P\left(|\overline{X} - \mathbb{E}[\overline{X}]| \geq \epsilon\right) \leq 2\exp\left(-\frac{2n^2\epsilon^2}{\sum_{i=1}^{n}(b_i - a_i)^2}\right)$.

20.2 Applying McDiarmid to GE Probability Bounds

Define g:

$$g(z_1, \ldots, z_n) = \sup_{f \in \mathcal{H}} (\mathcal{E}(f) - \mathcal{E}_n(f)).$$

as the worst generalization error. Then, apply McDiarmid, we get the GE Bound:

$$\sup_{f \in \mathcal{H}} (\mathcal{E}(f) - \mathcal{E}_n(f)) \leq \mathbb{E}\left[\sup_{f \in \mathcal{H}} (\mathcal{E}(f) - \mathcal{E}_n(f))\right] + c\sqrt{\frac{2\log(2/\delta)}{n}}.$$

***Derivation of GE Probability Bounds via McDiarmid** Need to read in detail later! **Step 1: Define the function** g. Let $(z_1, \ldots, z_n)$, where $z_i = (x_i, y_i)$, be the training dataset, define g as worst GE over $\mathcal{H}$:

$$g(z_1, \ldots, z_n) := \sup_{f \in \mathcal{H}} \left(\mathcal{E}(f) - \frac{1}{n} \sum_{i=1}^{n} \ell(f(x_i), y_i) \right),$$

Step 2: Verify the bounded difference condition. Let $z_i' = (x_i', y_i')$ be an alternative sample replacing z_i. Define:

$$g' := g(z_1, \ldots, z_{i-1}, z_i', z_{i+1}, \ldots, z_n).$$

We now bound the difference $|g - g'|$.

For any $f \in \mathcal{H}$, define:

$$A := \frac{1}{n}\sum_{j=1}^{n} \ell(f(x_j), y_j), \quad A' := \frac{1}{n}\sum_{j=1, j\neq i}^{n} \ell(f(x_j), y_j) + \frac{1}{n}\ell(f(x_i'), y_i').$$

Then:

$$|A - A'| = \left|\frac{1}{n}\left(\ell(f(x_i), y_i) - \ell(f(x_i'), y_i')\right)\right| \le \frac{1}{n} \cdot \left|\ell(f(x_i), y_i) - \ell(f(x_i'), y_i')\right|.$$

Assume the loss is uniformly bounded: $\ell(f(x), y) \in [0, c]$. Then:

$$|\ell(f(x_i), y_i) - \ell(f(x_i'), y_i')| \le 2c.$$

Should be c! But remember it as $2c$. Therefore,

$$|A - A'| \le \frac{2c}{n}.$$

Since this bound is uniform across all $f \in \mathcal{H}$, taking the supremum preserves the inequality:

$$|g(z_1, \ldots, z_n) - g(z_1, \ldots, z_i', \ldots, z_n)| \le \frac{2c}{n}.$$

Hence, g satisfies the bounded difference condition with constant $\frac{2c}{n}$.

Step 3: Apply McDiarmid's Inequality. From McDiarmid's theorem, for any $\delta \in (0,1)$, with probability at least $1-\delta$, plug in $\frac{2c}{n}$

$$|g(z_1, \ldots, z_n) - \mathbb{E}[g(z_1, \ldots, z_n)]| \le \sqrt{\frac{1}{2}\sum_{i=1}^{n}\left(\frac{2c}{n}\right)^2 \log\frac{2}{\delta}}.$$

Compute the RHS:

$$= \sqrt{\frac{n \cdot (4c^2)}{2n^2}\log\frac{2}{\delta}} = c\sqrt{\frac{2\log(2/\delta)}{n}}.$$

Step 4: Final High-Probability Bound. Thus, with probability at least $1-\delta$,

$$\sup_{f\in\mathcal{H}}\left(\mathcal{E}(f) - \mathcal{E}_n(f)\right) \le \mathbb{E}\left[\sup_{f\in\mathcal{H}}\left(\mathcal{E}(f) - \mathcal{E}_n(f)\right)\right] + c\sqrt{\frac{2\log(2/\delta)}{n}}.$$

Using the bound in terms of $\mathcal{R}_n(\mathcal{H})$, we get:

$$\sup_{f\in\mathcal{H}}\left(\mathcal{E}(f) - \mathcal{E}_n(f)\right) \le 2L\mathcal{R}_n(\mathcal{H}) + c\sqrt{\frac{2\log(2/\delta)}{n}}.$$

21 Caveats/Examples of Rademacher Complexity

$\mathcal{R}_n(\mathcal{H})$ has to be finite real number (doesn't mean $\mathcal{H}$ need to be finite), otherwise we get no generalization guarantee. In following examples, we check if $\mathcal{R}_n(\mathcal{H})$ is finite or not for common hypothesis spaces such as Unbounded linear, Bounded Linear, and Non-linear (RKHS).

21.1 1. Linear Hypotheses

21.1.1 Unbounded w

Consider $\mathcal{X} = \mathbb{R}^d$ and hypothesis space $\mathcal{H} = \{f(x) = \langle x, w\rangle \mid \forall x \in \mathcal{X}, \exists w \in \mathbb{R}^d\}$ with no norm bound on w. Then, $\mathcal{R}_n(\mathcal{H})$ is:

$$\mathcal{R}_n(\mathcal{H}) = \frac{1}{n}\mathbb{E}\left[\sup_{w\in\mathbb{R}^d}\sum_{i=1}^{n}\sigma_i\langle x_i, w\rangle\right].$$

By applying the Cauchy-Schwarz inequality, $\langle x, w\rangle \le \|x\|\|w\|$, this simplifies to:

$$\mathcal{R}_n(\mathcal{H}) \le \frac{1}{n}\mathbb{E}\left[\sup_{w}\langle w, \sum_{i=1}^{n}\sigma_i x_i\rangle\right] = \frac{1}{n}\mathbb{E}[\sup_{\|w\|}\|w\|\sum_{i=1}^{n}\|\sigma_i x_i\|] = \infty$$

without constraints on w, $\sup\|w\|$ can be infinite, thus $\mathcal{R}_n(\mathcal{H})$ is infinite too.

21.1.2 Bounded w

Now restrict w to a ball of radius γ:

$$\mathcal{H}_\gamma = \{f(x) = \langle x, w\rangle : \|w\| \leq \gamma\}$$

Then:

$$\mathcal{R}_n(\mathcal{H}_\gamma) = \frac{1}{n}\mathbb{E}\left[\sup_{\|w\|\leq\gamma}\left\langle w, \sum_{i=1}^{n}\sigma_i x_i\right\rangle\right] = \frac{\gamma}{n}\mathbb{E}\left[\left\|\sum_{i=1}^{n}\sigma_i x_i\right\|\right]$$

Using Jensen's inequality and assuming $\|x_i\| \leq B$:

$$\leq \frac{\gamma}{n}\sqrt{\mathbb{E}\left[\left\|\sum_{i=1}^{n}\sigma_i x_i\right\|^2\right]} = \frac{\gamma}{n}\sqrt{\sum_{i=1}^{n}\|x_i\|^2} \leq \frac{\gamma B\sqrt{n}}{n} = \frac{\gamma B}{\sqrt{n}}$$

$$\boxed{\mathcal{R}_n(\mathcal{H}_\gamma) \leq \frac{\gamma B}{\sqrt{n}}}$$

Hence, now GE bounds decays at rate $O(\frac{1}{\sqrt{n}})$ and controlled by weight norm γ and input norm B.

21.2 2. Reproducing Kernel Hilbert Spaces (RKHS)

Read it properly again later! Let $\mathcal{H}$ be a RKHS with kernel $k : \mathcal{X}\times\mathcal{X} \to \mathbb{R}$ such that **(bounded)**:

$$k(x, x') \leq \kappa^2, \quad \text{for all } x, x' \in \mathcal{X}.$$

Define the $\mathcal{H}_\gamma$ constrained by RKHS norm:

$$\mathcal{H}_\gamma = \{f \in \mathcal{H} : \|f\|_{\mathcal{H}} \leq \gamma\}.$$

Step 1: Use the Reproducing Property (IMPORTANT). In an RKHS, for every $f \in \mathcal{H}$:

$$f(x_i) = \langle f, k(x_i, \cdot)\rangle_{\mathcal{H}}.$$

i.e. the value of f at x is reproduced by the inner product with the kernel function centered at x. So we can rewrite:

$$\mathcal{R}_n(\mathcal{H}_\gamma) = \mathbb{E}_\sigma\left[\sup_{\|f\|_{\mathcal{H}}\leq\gamma}\left\langle f, \frac{1}{n}\sum_{i=1}^{n}\sigma_i k(x_i, \cdot)\right\rangle_{\mathcal{H}}\right].$$

Step 2: Apply Cauchy–Schwarz in RKHS.

$$\leq \mathbb{E}_\sigma\left[\sup_{\|f\|\leq\gamma}\|f\|_{\mathcal{H}}\cdot\left\|\frac{1}{n}\sum_{i=1}^{n}\sigma_i k(x_i, \cdot)\right\|_{\mathcal{H}}\right] = \gamma\cdot\mathbb{E}_\sigma\left[\left\|\frac{1}{n}\sum_{i=1}^{n}\sigma_i k(x_i, \cdot)\right\|_{\mathcal{H}}\right].$$

Step 3: Bound the RKHS Norm. To bound $\|\sum_{i=1}^{n}\sigma_i k(x_i, \cdot)\|_{\mathcal{H}}$, we compute the squared RKHS norm:

$$\left\|\sum_{i=1}^{n}\sigma_i k(x_i, \cdot)\right\|_{\mathcal{H}}^2 = \sum_{i=1}^{n}\sum_{j=1}^{n}\sigma_i\sigma_j\langle k(x_i, \cdot), k(x_j, \cdot)\rangle_{\mathcal{H}} = \sum_{i,j=1}^{n}\sigma_i\sigma_j k(x_i, x_j).$$

Taking expectation over σ:

$$\mathbb{E}_\sigma\left[\left\|\sum_{i=1}^{n}\sigma_i k(x_i, \cdot)\right\|_{\mathcal{H}}^2\right] = \sum_{i=1}^{n}\mathbb{E}_\sigma[\sigma_i^2]k(x_i, x_i) = \sum_{i=1}^{n}k(x_i, x_i) \leq n\kappa^2.$$

So plug the upper back,

$$\mathbb{E}_\sigma\left[\left\|\frac{1}{n}\sum_{i=1}^{n}\sigma_i k(x_i, \cdot)\right\|_{\mathcal{H}}\right] \leq \frac{1}{n}\sqrt{\mathbb{E}_\sigma\left[\left\|\sum_{i=1}^{n}\sigma_i k(x_i, \cdot)\right\|_{\mathcal{H}}^2\right]} \leq \frac{\kappa}{\sqrt{n}}.$$

where we used Jensen: $E[X] \leq \sqrt{E[X^2]}$ to link together.

$$\boxed{\mathcal{R}_n(\mathcal{H}_\gamma) \leq \frac{\gamma\kappa}{\sqrt{n}}.}$$

- γ controls the **hypothesis space size (norm constraint).**
- κ controls the **kernel complexity (e.g., similarity measure strength).**
- As n increases, generalization improves at the rate $O(1/\sqrt{n})$, $\mathcal{R}_n(\mathcal{H})$ is finite.

22 Constrained Optimization: $\mathcal{H}$

$\mathcal{H}$ can't be too rich as generalization will fail due to high $\mathcal{R}_n(\mathcal{H})$ (e.g. unbounded linear functions with large $\|w\|$). So instead, we optimize over **bounded** subsets of $\mathcal{H}$, i.e., **regularized or constrained** $\mathcal{H}$**.**)

22.1 1. Tikhonov Regularization

Given the same hypothesis space $\mathcal{H} = \{f(x) = \langle x, w\rangle \mid \forall x \in \mathcal{X}, \exists w \in \mathbb{R}^d\}$ as unbounded linear, we add a penalty on the ERM:

$$w_{S,\lambda} = \arg\min_{w\in\mathbb{R}^d} \mathcal{E}_S(w) + \lambda\|w\|^2$$

(only to empirical risk, to control the model complexity by tuning w, expected risk unknown) We write $\mathcal{E}_S(w)$ from now on to indicate the expected loss over linear function f with parameter w.

22.1.1 Bounding the Norm

Since $w_{S,\lambda}$ minimizes the regularized objective above,

$$\mathcal{E}_S(w_{S,\lambda}) + \lambda\|w_{S,\lambda}\|^2 \le \mathcal{E}_S(0)$$

Assuming loss is bounded: $\ell(y', y) \le M^2$, we have:

$$\lambda\|w_{S,\lambda}\|^2 \le \mathcal{E}_S(0) \le \frac{1}{n}\sum_{j=1}^{n} \ell(0, y_j) \le M^2 \quad \Rightarrow \quad \|w_{S,\lambda}\| \le \frac{M}{\sqrt{\lambda}}$$

So $w_{S,\lambda} \in \mathcal{H}$, has norm bound $\gamma = \frac{M}{\sqrt{\lambda}}$, ensuring **finite** $\mathcal{R}_n(\mathcal{H})$. (This is an example where regularization + unbounded linear = finite $\mathcal{R}_n(\mathcal{H})$).

22.2 2. Excess Risk Decomposition for Regularized ERM

Let w_* be the true risk $\mathcal{E}(w)$ minimizer, and $w_{S,\lambda}$ is empirical risk minimizer. Then:

$$\mathbb{E}[\mathcal{E}(w_{S,\lambda}) - \mathcal{E}(w_*)] = \underbrace{\mathbb{E}[\mathcal{E}(w_{S,\lambda}) - \mathcal{E}_S(w_{S,\lambda})]}_{\text{Generalization Error}} + \underbrace{\mathbb{E}[\mathcal{E}_S(w_{S,\lambda}) - \mathcal{E}_S(w_*)]}_{\le\lambda\|w_*\|^2} + \underbrace{\mathbb{E}[\mathcal{E}_S(w_*) - \mathcal{E}(w_*)]}_{=0}$$

Using generalization bound from Rademacher complexity:

$$\mathbb{E}[\mathcal{E}(w_{S,\lambda}) - \mathcal{E}(w_*)] \le 2L\mathcal{R}_n(\mathcal{H}_\gamma) + \lambda\|w_*\|^2 \le \frac{2LMB}{\sqrt{n\lambda}} + \lambda\|w_*\|^2$$

where we use $\mathcal{R}_n(\mathcal{H}_\gamma) \le \frac{MB}{\sqrt{n}}$ bound derived from bounded linear space.

Choosing optimal $\lambda(n) = \dfrac{(LMB)^{2/3}}{\|w_*\|^{4/3}n^{1/3}}$ (Minimizer of the upper bound $\frac{2LMB}{\sqrt{n\lambda}} + \lambda\|w_*\|^2$), we get the final excess risk upper bound:

$$\mathbb{E}[\mathcal{E}(w_{S,\lambda(n)}) - \mathcal{E}(w_*)] \le \frac{3(LMB)^{2/3}\|w_*\|^{2/3}}{n^{1/3}}$$

22.3 3. Ivanov Regularization

Instead of adding penalty to the norm w objective function, Ivanov regularization directly constrains via $\|w\| \le \gamma$:

$$w_{S,\gamma} = \arg\min_{\|w\|\le\gamma} \mathcal{E}_S(w)$$

Similar to how we derived the norm bound $\gamma = \frac{M}{\sqrt{\lambda}}$ in Tikhonov regularization, also how we bound the norm in Bounded Linear Hypotheses in last section!

This method gives:

$$\mathbb{E}[\mathcal{E}(w_{S,\gamma}) - \mathcal{E}(w_*)] \leq O\left(\frac{1}{\sqrt{n}}\right)$$

***Derivation of Excess Risk Upper Bound via Ivanov Regularization** Let w_* be the optimal predictor from true distribution. Then:

$$\mathbb{E}[\mathcal{E}(w_{S,\lambda}) - \mathcal{E}(w_*)] = \underbrace{\mathbb{E}[\mathcal{E}(w_{S,\lambda}) - \mathcal{E}_S(w_{S,\lambda})]}_{\text{Generalization Error}} + \underbrace{\mathbb{E}[\mathcal{E}_S(w_{S,\lambda}) - \mathcal{E}_S(w_*)]}_{\leq 0} + \underbrace{\mathbb{E}[\mathcal{E}_S(w_*) - \mathcal{E}(w_*)]}_{=0}$$

Bound First Term: Since $\|w_{S,\gamma}\| \leq \gamma$, the hypothesis space is the ball $\mathcal{H}_\gamma = \{w : \|w\| \leq \gamma\}$. Using the Rademacher complexity bound for linear hypotheses (we derived above both expectation bound $2L \cdot \mathcal{R}_n(\mathcal{H}_\gamma)$ and $\mathcal{R}_n(\mathcal{H}_\gamma) \leq \frac{\gamma B}{\sqrt{n}}$ respectively) :

$$\mathbb{E}[\mathcal{E}(w_{S,\gamma}) - \mathcal{E}_S(w_{S,\gamma})] \leq 2L \cdot \mathcal{R}_n(\mathcal{H}_\gamma) \leq 2L \cdot \frac{\gamma B}{\sqrt{n}}$$

Bound Second Term: Since $w_{S,\gamma}$ is the empirical risk minimizer under constraint, while w_* is the true risk minimizer under constraint:

$$\mathcal{E}_S(w_{S,\gamma}) \leq \mathcal{E}_S(w_*) \quad \Rightarrow \quad \mathcal{E}_S(w_{S,\gamma}) - \mathcal{E}_S(w_*) \leq 0$$

So this term is non-positive and can be dropped in the upper bound. Overall, excess risk is bounded by:

$$\mathbb{E}[\mathcal{E}(w_{S,\gamma}) - \mathcal{E}(w_*)] \leq 2L \cdot \frac{\gamma B}{\sqrt{n}} = O\left(\frac{1}{\sqrt{n}}\right)$$

22.4 4. Projected Gradient Descent (PGD)

When using Ivanov-style constraints $\|w\| \leq \gamma$, can use PGD to solve the constrained ERM $\min_{\|w\| \leq \gamma} \mathcal{E}_S(w)$ by finding the optimal w:

$$w_{k+1} = \Pi_C(w_k - \eta \nabla F(w_k))$$

where Π_C denotes projection onto the constraint set C and $F(w) = \mathcal{E}_{\mathcal{S}}(\sqsupseteq)$ is empirical risk..

Let function F be M-smooth and convex. Then after K steps the rate is $O(1/K)$:

$$F(w_K) - F(w^*) \leq \frac{M}{2K}\|w_0 - w^*\|^2$$

Combining with the $O(1/\sqrt{N})$ generalization error, can trade off computation K and sample size n to get overall guarantee.

23 Online Learning

23.1 Having Algorithm

Input:

n experts making predictions $x_{t,i} \in \{0, 1\}$ of expert i at time t.
True outcome $y_t \in \{0, 1\}$ at each round t.
Assume a perfect expert i^* exists such that:
$x_{t,i^*} = y_t, \quad \forall t \in \{1, \ldots, m\}$.

Initialize:

$\mathcal{C}_1 = \{1, 2, \ldots, n\}$ (All n experts are consistent initially.)

For each round $t = 1, \ldots, m$:

1. **Predict:**
   Compute the **majority vote** among consistent experts:
   $\hat{y}_t = \begin{cases} 1 & \sum_{i \in \mathcal{C}_t} x_{t,i} \geq \frac{|\mathcal{C}_t|}{2} \\ 0 & \sum_{i \in \mathcal{C}_t} x_{t,i} < \frac{|\mathcal{C}_t|}{2} \end{cases}$
2. **Update Consistent Experts:**
   Remove inconsistent experts from $\mathcal{C}_t$ based on the true outcome y_t:
   $\mathcal{C}_{t+1} = \{i \in \mathcal{C}_t : x_{t,i} = y_t\}$.
3. **Repeat for the next round.**

Output:

Cumulative loss of the algorithm:
$L_A(S) = \sum_{t=1}^{m} |\hat{y}_t - y_t|$.

***Derivation of Having Algorithm Mistake Bound**
Initialize with $|\mathcal{C}_1| = n$ consistent experts. Each mistake eliminates at least half of the consistent experts:

$$|\mathcal{C}_{t+1}| \leq \frac{|\mathcal{C}_t|}{2}.$$

After M mistakes, the number of consistent experts is:

$$|\mathcal{C}_{M+1}| \leq \frac{n}{2^M}.$$

To ensure at least one consistent expert remains (the perfect expert), we require:

$$\frac{n}{2^M} \geq 1.$$

Solving for M to obtain the mistake bound:

$$M \leq \log_2(n).$$

23.1.1 Extension to Imperfect Experts

If no perfect expert exists, the best expeert is the one with the lowest cumulative loss:

$$L_{i^*}(S) = \sum_{t=1}^{m} |y_t - x_{t,i^*}|.$$

where $L_i(S)$ is the total loss of expert i on sequence S. Then, the cumulative loss is bounded by:

$$L_A(S) \leq a \cdot \min_i L_i(S) + b \cdot \log(n).$$

where $\min_i L_i(S)$ is the loss of the best expert and a, b are small constants. We've derived that the **number of mistakes is bounded by** $\log_2(n)$, but now we must account for the errors of the best expert too as additional cost.

Aspect	**WMA**	**WAA**	**Hedge**
Expert prediction $x_{t,i}$	$\{0,1\}$ (binary labels)	$[0,1]$ (probabilities)	$[0,1]$ *losses* $\ell_{t,i}$ of n *actions*
Master prediction $\hat{y}_t$	$\{0,1\}$ (weighted majority vote)	$[0,1]$ (weighted average)	Distribution $v_t \in \Delta^n$ as vector of n
Master prediction rule	$\hat{y}_t = \mathbb{I}\Big[\sum_i w_{t,i}x_{t,i} \geq \frac{W_t}{2}\Big]$ $(W_t = \sum_{i=1}^{n} w_{t,i})$	$\hat{y}_t = \sum_i v_{t,i}x_{t,i}, \quad v_{t,i} = w_{t,i}/W_t$	Loss incurred is $v_t \ell_t$ (HA-1) or $\ell_{t,\iota}$ with $\iota \sim v_t$ (HA-2)
Weight initialisation	$w_{1,i} = 1$	$w_{1,i} = 1$	$w_{1,i} = 1$
Weight-update formula	$w_{t+1,i} = w_{t,i}\, \beta^{\mathbb{I}\{x_{t,i} \neq y_t\}}$ $(\beta \in [0,1)$	$w_{t+1,i} = w_{t,i}\, e^{-\eta\, L(y_t, x_{t,i})}, \quad \eta > 0$	$w_{t+1,i} = w_{t,i}\, e^{-\eta\, \ell_{t,i}}, \quad \eta > 0$
Normalisation step	None	$v_{t,i} = w_{t,i}/\sum_j w_{t,j}$	$v_{t,i} = w_{t,i}/\sum_j w_{t,j}$
Loss	$L_A(S) = \sum_{t=1}^{m} |\hat{y}_t - y_t|$.	Any bounded convex loss $L : \mathcal{Y} \times \hat{\mathcal{Y}} \to [0,1]$	$L_A(t) = v_t \cdot \ell_t$ where $\ell_t \in [0,1]^n$
Learning-rate / penalty	β (typical choice $\beta = e^{-1}$)	η (tune as $\eta = \sqrt{\frac{\ln n}{mB^2}}$ for B-bounded loss)	$\eta = \sqrt{\frac{\ln n}{2m}}$ (optimal for $[0,1]$ losses)
Regret / mistake bound	$M \leq \frac{\ln\frac{1}{\beta}}{\ln\frac{2}{1+\beta}} M_{i^\star} + \frac{\ln n}{\ln\frac{2}{1+\beta}}$ $(\beta = e^{-1} \Rightarrow M \leq 2.63\, M_{i^\star} + 2.63 \ln n)$	$L_{WA}(S) - \min_i L_i(S) \leq \sqrt{2m \ln n}$ for $\ell \in [0,1]$	$\mathbb{E}[L_{HA}] - \min_i L_i \leq \sqrt{2m \ln n}$
Elimination of experts	Hard elimination when wrong majority	**Never eliminated** (weights can vanish)	**Never eliminated** (weights can vanish)
Prediction type supported	Binary classification	Probabilistic regression	Online portfolio / adversarial bandits
Average regret convergence	$M/m \to 0$ if best expert mistake rate < 1	$\big(L_{WA} - \min_i L_i\big)/m \to 0$	$\big(\mathbb{E}[L_{HA}] - \min_i L_i\big)/m \to 0$

23.2 1. Weighted Majority Algorithm (WMA)

***Derivation of WMA Mistake Bound**

Step 1: Weight Evolution Analysis The weight of each expert i at round t is given by (cumulative version of $w_{t+1,i} = w_{t,i} \times \beta^{\mathbb{I}\{x_{t,i} \neq y_t\}}$) :

$$w_{t,i} = \beta^{M_{t,i}}$$

where $M_{t,i}$ is the number of mistakes made by expert i up to round t.

The total weight of all experts at the start of round t is **(unnormalized)**:

$$W_t = \sum_{i=1}^{n} w_{t,i} = \sum_{i=1}^{n} \beta^{M_{t,i}}$$

Step 2: Weight Decay on Mistake of Master Algorithm If the master algorithm makes a mistake at round t, the weights of the majority experts (who were wrong) are scaled by β:

$$W_{t+1} = (\text{Minority}) + \beta \times (\text{Majority}) \leq \frac{W_t}{2} + \beta \times \frac{W_t}{2} = \frac{1+\beta}{2} W_t$$

Step 3: Recursive Decay over m Rounds We apply this recursive decay over all M rounds (i.e. M mistakes):

$$W_{m+1} \leq \left(\frac{1+\beta}{2}\right)^M W_1$$

With initial total weight $W_1 = n$, this becomes:

$$W_{m+1} \leq \left(\frac{1+\beta}{2}\right)^M n$$

Step 4: Relating to Best Expert Weight Since the weight of the best expert i^* is (M_i* is the total mistakes of best expert):

$$w_{m+1,i^*} = \beta^{M_{i^*}}$$

So W_{t+1} as total weight of all experts, it upper bound best expert:

$$\left(\frac{1+\beta}{2}\right)^M n \geq \beta^{M_{i^*}}$$

Step 5: Solving for M We solve for the regret bound of M **(Total Mistakes of the Master)**:

$$M \ln\left(\frac{1+\beta}{2}\right) + \ln n \geq M_{i^*} \ln \beta$$

$$M \leq \frac{\ln \frac{1}{\beta}}{\ln \frac{2}{1+\beta}} M_{i^*} + \frac{1}{\ln \frac{2}{1+\beta}} \ln n$$

Step 6: Special Case with $\beta = \frac{1}{e}$ We can simplify the bound by setting $\beta = \frac{1}{e}$:

$$M \leq 2.63 \min_i M_i + 2.63 \ln n$$

So we can summarize that WMA achieves a regret bound of:

$$L_A(S) \leq 2.63 \min_i L_i(S) + 2.63 \ln n$$

which means the performance of WMA is close to the scaled best expert's performance plus a small term that scales with $\ln n$.

23.3 2. Weighted Average Algorithm (WAA)

Both Halving and WMA assume 1. binary prediction 2. cumulative error loss. Now we generalize to arbitrary loss: $L : \mathcal{Y} \times \hat{\mathcal{Y}} \to [0, 1]$.

Algorithm 3 Exponentially Weighted Average Forecaster (WAA)

Initialize:

- Set weights $w_{1,i} = 1$ for all experts $i = 1, \dots, n$.
- Set normalized weights $v_{1,i} = w_{1,i} / \sum_j w_{1,j} = \frac{1}{n}$.

Input: learning rate $\eta > 0$, loss $L : \mathcal{Y} \times \mathcal{Y} \to [0, 1]$.

for $t = 1$ to m **do**

  Receive expert predictions $x_t = (x_{t,1}, \dots, x_{t,n}) \in [0, 1]^n$.

  Predict

$$\hat{y}_t = \sum_{i=1}^{n} v_{t,i} \, x_{t,i}.$$

  Observe true outcome $y_t \in [0, 1]$ and incur loss $L(y_t, \hat{y}_t)$.

  Update expert weights:

$$w_{t+1,i} = w_{t,i} \exp\bigl(-\eta \, L(y_t, x_{t,i})\bigr), \quad v_{t+1,i} = \frac{w_{t+1,i}}{\sum_{j=1}^{n} w_{t+1,j}}.$$

end for

23.4 3. Hedge Algorithm

Algorithm 4 Hedge Algorithm

Initialize: $w_{1,i} = 1$ for $i = 1, \dots, n$.

Input: learning rate $\eta > 0$.

for $t = 1$ to m **do**

  Form distribution $v_{t,i} = \frac{w_{t,i}}{\sum_j w_{t,j}}$.

  Receive loss vector $\ell_t = (\ell_{t,1}, \dots, \ell_{t,n}) \in [0, 1]^n$.

  Incur expected loss $v_t \cdot \ell_t = \sum_i v_{t,i} \, \ell_{t,i}$.

  Update weights:

$$w_{t+1,i} = w_{t,i} \exp\bigl(-\eta \, \ell_{t,i}\bigr).$$

end for

Regret Bounds for WAA vs. Hedge

1. WAA with exp-concave losses

- **Assumption:** $L(y, \hat{y})$ is α-exp-concave in $\hat{y}$, such as
  - Entropic loss $L(y, \hat{y}) = y \ln \frac{y}{\hat{y}} + (1 - y) \ln \frac{1-y}{1-\hat{y}}$ **(1-exp-concave)**
  - Squared loss $L(y, \hat{y}) = (y - \hat{y})^2$ **($\frac{1}{2}$-exp-concave)**
- **Regret Bound:**

$$L_{WA}(1\!:\!m) - \min_i L_i(1\!:\!m) \le \frac{\ln n}{\eta} \quad \text{(independent of } m\text{)}.$$

  - Choose $\eta = 1$ for entropic loss, $\eta = 0.5$ for squared loss.

2. WAA/Hedge with losses in $[0, 1]$

- **Assumption:** each per-round loss $\ell_{t,i} \in [0, 1]$, no exp-concavity or other assumption.
- **Regret Bound:**

$$L_{WA}(1\!:\!m) - \min_i L_i(1\!:\!m) \le \frac{\ln n}{\eta} + \frac{\eta \, m}{2}.$$

- **Optimizing** $\eta = \sqrt{\frac{2\ln n}{m}}$ yields

$$L_{WA} - \min_i L_i \;\le\; \sqrt{2\,m\ln n}.$$

3. Extension to losses in $[0, B]$

- **Assumption:** each $\ell_{t,i} \in [0, B]$.
- **Regret Bound:**

$$L_{WA} - \min_i L_i \;\le\; \frac{\ln n}{\eta} + \frac{\eta\, B^2\, m}{2} \quad\Longrightarrow\quad B\sqrt{2\,m\ln n}.$$

***Example: Derive Regret Bound for Entropic Loss**

1. Exponential–weight setup With multiplicative updates

$$w_{t+1,i} = w_{t,i}\, e^{-\eta\, L(y_t, x_{t,i})}, \qquad w_{1,i} = 1,$$

$$\boxed{w_{t,i} = e^{-\eta\, L_{t,i}}}, \qquad L_{t,i} := \sum_{s=1}^{t} L(y_s, x_{s,i}).$$

The aggregate weight is

$$W_t := \sum_{i=1}^{n} w_{t,i}, \qquad \text{so } W_1 = n.$$

2. Master-algorithm loss vs. weights Because the entropic loss is ~~η~~-mixable / convex: $(L(y, \sum vx) \le \sum v\, L(y, x).)$

$$L\Big(y_t, \hat{y}_t\Big) = L\Big(y_t, \sum_i v_{t,i} x_{t,i}\Big) \;\le\; \sum_i v_{t,i} L(y_t, x_{t,i}), \qquad v_{t,i} := \frac{w_{t,i}}{W_t}.$$

Summing over t and inserting $v_{t,i}$:

$$\boxed{L_A(S) = \sum_{t=1}^{m} L(y_t, \hat{y}_t) \le \frac{1}{\eta}\big[\ln W_1 - \ln W_{m+1}\big]}. \tag{1}$$

3. Lower bound on the final weight For the best expert i^*:

$$w_{m+1,i^*} = e^{-\eta\, L_{i^*}(S)} \quad\Longrightarrow\quad \boxed{W_{m+1} \ge e^{-\eta\, L_{i^*}(S)}} \quad\Longrightarrow\quad -\ln W_{m+1} \le \eta\, L_{i^*}(S). \qquad ()$$

4. Combine () and () Insert () into () and note $W_1 = n$:

$$L_A(S) \;\le\; \frac{1}{\eta}\ln n + L_{i^*}(S).$$

5. Optimal learning rate Setting $\eta = 1$ (standard for log-loss):

$$\boxed{L_A(S) - \min_i L_i(S) \le \ln n}.$$

***Derivation of Hedge Regret Bound.** Read this in detail later, looks so complex...
We first establish the following **progress versus regret inequality**:

$$v_t \cdot \ell_t - u \cdot \ell_t \le \frac{1}{\eta}\left(d(u, v_t) - d(u, v_{t+1})\right) + \frac{\eta}{2}\sum_{i=1}^{n} v_{t,i}\ell_{t,i}^2, \quad \forall u \in \Delta_n.$$

Step 1: Definition of Weight Update Rule Define the normalizing constant:

$$Z_t := \sum_{i=1}^{n} v_{t,i}\exp(-\eta \ell_{t,i}),$$

which ensures that the updated weights satisfy:

$$v_{t+1,i} = \frac{v_{t,i}\exp(-\eta\ell_{t,i})}{Z_t}.$$

Step 2: Change in Bregman Divergence The change in Bregman divergence is given by:

$$d(u, v_t) - d(u, v_{t+1}) = \sum_{i=1}^{n} u_i \ln \frac{v_{t+1,i}}{v_{t,i}}.$$

Expanding the logarithm:

$$= -\eta \sum_{i=1}^{n} u_i \ell_{t,i} - \sum_{i=1}^{n} u_i \ln Z_t.$$

Using Jensen's inequality on Z_t, we obtain:

$$-\sum_{i=1}^{n} u_i \ln Z_t \geq -\ln \sum_{i=1}^{n} v_{t,i} e^{-\eta \ell_{t,i}}.$$

Applying the bound $e^{-x} \leq 1 - x + \frac{x^2}{2}$ for $x \geq 0$, we approximate:

$$\sum_{i=1}^{n} v_{t,i} e^{-\eta \ell_{t,i}} \leq \sum_{i=1}^{n} v_{t,i} (1 - \eta \ell_{t,i} + \frac{1}{2} \eta^2 \ell_{t,i}^2).$$

Thus, using $\ln(1 + x) \leq x$:

$$d(u, v_t) - d(u, v_{t+1}) \geq \eta (v_t \cdot \ell_t - u \cdot \ell_t) - \frac{1}{2} \eta^2 \sum_{i=1}^{n} v_{t,i} \ell_{t,i}^2.$$

Rearranging yields the desired inequality.

Step 3: Summing Over All Rounds Summing over t from 1 to m, we obtain:

$$\sum_{t=1}^{m} (v_t \cdot \ell_t - u \cdot \ell_t) \leq \frac{1}{\eta} (d(u, v_1) - d(u, v_{m+1})) + \frac{\eta}{2} \sum_{t=1}^{m} \sum_{i=1}^{n} v_{t,i} \ell_{t,i}^2.$$

Using the bounds:

$$d(u, v_1) \leq \ln n, \quad -d(u, v_{m+1}) \leq 0, \quad \sum_{t=1}^{m} \sum_{i=1}^{n} v_{t,i} \ell_{t,i}^2 \leq m,$$

we optimize η by setting:

$$\eta = \sqrt{\frac{2 \ln n}{m}},$$

which gives the final regret bound:

$$\mathbb{E}[L_{WA}(S)] - \min_i L_i(S) \leq \sqrt{2m \ln n}.$$

(When extending to loss $\in [0, B]$, the learning rate changes to $\eta = \sqrt{\frac{2 \ln n}{m B^2}}$ and the final regret bound is:

$$\mathbb{E}[L_{WA}(S)] - \min_i L_i(S) \leq B\sqrt{2m \ln n}.$$

24 Perceptron Algorithm

24.1 Algorithm

Algorithm 5 Perceptron

Initialize: $w_1 = 0 \in \mathbb{R}^n$.
Input: (x_t, y_t), $x_t \in \mathbb{R}^n$, $y_t \in \{-1, +1\}$.

```
for t = 1, ..., T do
    Predict ŷ_t = sign(w_t · x_t).
    if y_t (w_t · x_t) ≤ 0 then
        (i.e. Mistake occurs)Update the weights: w_{t+1} = w_t + y_t x_t.
    else
        w_{t+1} = w_t.
    end if
end for
```

24.2 Perceptron Mistake Bound

Assume if data is linearly separable, there exists weight vector u with $\|u\| = 1$ and $y_t(u \cdot x_t) \geq \gamma > 0$, $\|x_t\| \leq R$. Then

$$M \ \leq \ \left(\tfrac{R}{\gamma}\right)^2.$$

***Derivation of Perceptron Mistake Bound.**

1. **Growth of the Inner Product with u**: At each mistake t, the update increases the alignment of w_t with u:

$$w_{t+1} \cdot u = w_t \cdot u + y_t(x_t \cdot u).$$

   Given separability assumption $y_t(u \cdot x_t) \geq \gamma$,

$$w_{t+1} \cdot u \geq w_t \cdot u + \gamma.$$

   By induction over the M mistakes, we get:

$$w_{M+1} \cdot u \geq M\gamma.$$

2. **Bounding the Norm of w_t**: Each mistake also increases the norm of w_t:

$$\|w_{t+1}\|^2 = \|w_t\|^2 + \|x_t\|^2 + 2y_t(w_t \cdot x_t).$$

   Since $y_t(w_t \cdot x_t) \leq 0$ (due to being a mistake), we have:

$$\|w_{t+1}\|^2 \leq \|w_t\|^2 + \|x_t\|^2.$$

   Using $\|x_t\| \leq R$, after M mistakes:

$$\|w_{M+1}\|^2 \leq MR^2.$$

3. **Combining the Bounds**: Using the Cauchy-Schwarz inequality:

$$w_{M+1} \cdot u \leq \|w_{M+1}\|\|u\|,$$

   and substituting the bounds:

$$M\gamma \leq \sqrt{MR^2}.$$

$$M \leq \left(\frac{R}{\gamma}\right)^2.$$

   Notes: The final weight vector w_{M+1} isn't guaranteed to maximize the margin, but simply a valid separator.

25 Online Gradient Descent (OGD) for Hinge Loss

Hinge loss:

$$\ell_t(w) = \max\{0, \, 1 - y_t \, (w \cdot x_t)\}.$$

25.1 Algorithm

Algorithm 6 OGD with Hinge Loss

Input: step-size $\eta > 0$ **Initialize:** $w_1 \;=\; 0 \in \mathbb{R}^n$, $L_{\text{OGD}} := 0$.

for $t = 1, \ldots, , m$ **do**
  Receive instance $x_t \in \mathbb{R}^n$.
  Predict margin:
$$m_t \;=\; w_t \cdot x_t, \quad \hat{y}_t = (m_t).$$
  Receive true label $y_t \in \{-1, +1\}$.
  Compute hinge loss:
$$\ell_t \;=\; \max\{0,\; 1 - y_t\, m_t\}.$$
  Accumulate loss:
$$L_{\text{OGD}} \mathrel{+}= \; \ell_t.$$
  Update weights:
$$w_{t+1} \;=\; \begin{cases} w_t + \eta\, y_t\, x_t, & y_t\, m_t < 1, \\ w_t, & y_t\, m_t \geq 1. \end{cases}$$
end for
return final weight w_{T+1} and total hinge loss L_{OGD}.

25.2 OGD Regret Bound

Enrich with more details later, esp the first step. In OGD setting, we compare the online sequence $\{w_t\}$ to any fixed comparator u. By the standard OGD analysis one shows

$$R_T(u) = \sum_{t=1}^{T} \ell_t(w_t) - \sum_{t=1}^{T} \ell_t(u) \;\leq\; \frac{\|u\|^2}{2\eta} \;+\; \frac{\eta}{2} \sum_{t=1}^{T} \|\nabla \ell_t(w_t)\|^2.$$

For hinge loss $\ell_t(w) = \max\{0, 1 - y_t(w \cdot x_t)\}$,

$$\|\nabla \ell_t(w_t)\| \leq \|x_t\| \leq R.$$

Hence,

$$R_T(u) \;\leq\; \frac{\|u\|^2}{2\eta} \;+\; \frac{\eta\, R^2\, T}{2}.$$

Optimizing $\eta = \frac{\|u\|}{R\sqrt{T}}$ gives the familiar

$$R_T(u) \;\leq\; R\, \|u\|\, \sqrt{T}.$$

25.2.1 Extension to losses in $[0, B]$

If each $\ell_t(w) \in [0, B]$, then $\|\nabla \ell_t\| \leq B\, R$ and the same argument yields

$$R_T(u) \;\leq\; \frac{\|u\|^2}{2\eta} \;+\; \frac{\eta\, B^2 R^2\, T}{2} \quad\Longrightarrow\quad R_T(u) \;\leq\; B\, R\, \|u\|\, \sqrt{T}$$

for $\eta = \dfrac{\|u\|}{B\, R\sqrt{T}}$.

25.3 Regularised OGD

Replace pure hinge loss in OGD with regularized version:

$$\ell_t^{\text{reg}}(w) = \frac{\lambda}{2} \|w\|^2 + \max\{0,\; 1 - y_t(w \cdot x_t)\}, \quad \lambda > 0.$$

to prevent $\|w\|$ from growing without bound while λ trades off margin-violations with model complexity.

Running OGD with step-size $\eta = 1$ on ℓ_t^{reg} yields the **regret bound**:

$$R_T^{\text{reg}} = \sum_{t=1}^{T} \ell_t^{\text{reg}}(w_t) - \min_w \sum_{t=1}^{T} \ell_t^{\text{reg}}(w) \;\leq\; \frac{1}{2\lambda} \;+\; \sqrt{T}.$$

Intuitively, the regularizer stabilizes the weights so you pay less regret for large T.

26 Kernelized Online Perceptron

Maintain coefficients $\{\alpha_s\}_{s\in}$ where indexes mistaken rounds. The classifier:

$$w = \sum_{s\in} \alpha_s\, y_s\, \phi(x_s), \quad f(x) = \big(\sum_{s\in} \alpha_s\, y_s\, k(x_s, x)\big).$$

26.1 Mistake Bound in RKHS

If $\|\phi(x_t)\| \le R$, margin $y_t \langle u, \phi(x_t)\rangle \ge \gamma$ in RKHS, then

$$M \;\le\; \left(\tfrac{R}{\gamma}\right)^2$$

identical to the linear case but in feature space.

	Perceptron	**OGD (Hinge)**	**OGD (Reg.)**
Update rule	$w_{t+1} = w_t + y_t x_t$ if $y_t(w_t \cdot x_t) \le 0$	$w_{t+1} = w_t - \eta g_t,\ g_t \in \partial \ell_t$	$w_{t+1} = w_t - \big(\lambda w_t + g_t\big)$
Loss used	0–1 loss (implicit)	hinge loss $\max\{0, 1 - y_t(w \cdot x_t)\}$	$\frac{\lambda}{2}\|w\|^2 + \max\{0, 1 - y_t(w \cdot x_t)\}$
Performance guarantee	Mistakes $M \le (R/\gamma)^2$	Regret $R_T \le U\, R\sqrt{T}$	Regret $R_T^{\text{reg}} \le \frac{1}{2\lambda} + \sqrt{T}$
Key assumption	Data separable with margin γ	Convex loss, $\|u\| \le U$, $\|x\| \le R$	Strong convexity via λ, $\|x\| \le R$
Per-iter. cost	$O(n)$ vector add	$O(n)$ gradient step	$O(n)$ gradient + reg. term
Memory	$O(n)$	$O(n)$	$O(n)$

27 Structured Learning

27.1 Surrogate Method Framework

- **Training Data:** Given a dataset $\{(x_i, y_i)\}_{i=1}^n$, with output space $\mathcal{Y}$ and input space $\mathcal{X}$.
- **Encoding:** Choose a feature map
$$c : \mathcal{Y} \longrightarrow \mathcal{H},$$
where $\mathcal{H}$ is a **real vector linear space** (e.g., $\mathcal{H} = \mathbb{R}$).
[If $\mathcal{Y}$ has many parts, the encoding concatenates into one long vector such as one-hot.]
- **Surrogate Loss Minimization:** Solve:
$$g_n = \arg\min_{g \in \mathcal{G}} \frac{1}{n} \sum_{i=1}^{n} L\big(g(x_i),\, c(y_i)\big),$$
where:
  - $\mathcal{G} \subseteq \{\mathcal{X} \to \mathcal{H}\}$ is a (convex) hypothesis class.
  - $L : \mathcal{H} \times \mathcal{H} \to \mathbb{R}$ is a **convex surrogate loss** (e.g., Squared loss).
- **Decoding:** Ensure the final model is written as:
$$d : \mathcal{H} \longrightarrow \mathcal{Y}, \qquad f_n(x) = d\big(g_n(x)\big).$$
[An example of d is $\text{argmax}_y \langle g_n(x), c(y)\rangle$]
- **Return:** The final predictor f_n.

Multi-class Classification Example

- **Training Data:** Given a dataset $\{(x_i, y_i)\}_{i=1}^n$, where each label $y_i \in \{1, \ldots, T\}$.
- **Encoding:** Use one-hot vectors to map each label t to a standard basis vector:

$$c : \{1, \ldots, T\} \to \mathbb{R}^T, \quad c(t) = e_t, \quad e_t^{(j)} = \begin{cases} 1, & j = t; \\ 0, & j \neq t. \end{cases}$$

  [This embedding turns the discrete label t into a point in linear space $\mathbb{R}^T$, allowing convex losses.]

- **Hypothesis Class:** Define a linear hypothesis class:

$$\mathcal{G} = \{ g : \mathcal{X} \to \mathbb{R}^T \mid g(x) = W^\top \phi(x),\ W \in \mathbb{R}^{d \times T} \},$$

  - $W \in \mathbb{R}^{d \times T}$ is the weight matrix, with each column $w_t \in \mathbb{R}^d$ scoring class t.
  - $\phi : \mathcal{X} \to \mathbb{R}^d$ is a feature map of the input.
  - [Each coordinate $g(x)^t$ is a real-valued "score" for class t. We will optimize W so that the correct class's score is highest.]

- **Surrogate Loss Minimization:** Learn the weights W_n by solving:

$$W_n = \arg \min_{W \in \mathbb{R}^{d \times T}} \frac{1}{n} \sum_{i=1}^n \sum_{t=1}^T \ell\big((W^\top \phi(x_i))^{(t)},\ e_{y_i}^{(t)}\big),$$

  - $\ell : \mathbb{R} \times \{0, 1\} \to \mathbb{R}$ is a convex loss function, such as:
    - **Hinge loss:** $\ell(s, 1) = \max(0, 1 - s)$, $\ell(s, 0) = \max(0, 1 + s)$.
    - **Logistic loss:** $\ell(s, 1) = \ln(1 + e^{-s})$, $\ell(s, 0) = \ln(1 + e^s)$.
  - The optimization is typically performed using gradient descent.

- **Decoding:** For any input $x \in \mathcal{X}$, after learning W_n, we predict the label as:

$$f_n(x) = \arg \max_{t=1,\ldots,T} \big(W_n^\top \phi(x)\big)^{(t)}.$$

  [This means choosing the class t whose score $w_t^\top \phi(x)$ is largest.]

- **Return:** The final multi-class classifier f_n.

27.2 Likelihood Estimation

Directly models $P(y \mid x)$ even when y has complex structure:

Step 1: Approximation. Define a real-valued score (or un-normalized log-probability)

$$P_n(y \mid x) = w_n^\top \psi(y, x), \quad (y, x) \in \mathcal{Y} \times \mathcal{X}.$$

where joint feature map $\psi : \mathcal{Y} \times \mathcal{X} \ \to\ \mathbb{R}^d$.

Step 2: Prediction (Decoding). Given a new input x, predict

$$\hat{f}(x) = \arg \max_{y \in \mathcal{Y}}\ P_n(y \mid x) = \arg \max_{y \in \mathcal{Y}}\ w_n^\top \psi(y, x).$$

Training. Optimize w_n by maximizing conditional likelihood:

$$w_n = \arg \max_{w \in^d} \sum_{i=1}^n \ln \frac{\exp\big(w^\top \psi(y_i, x_i)\big)}{\sum_{y' \in \mathcal{Y}} \exp\big(w^\top \psi(y', x_i)\big)} \ -\ \lambda \|w\|^2,$$

27.3 Implicit Loss Embedding

*A loss function $\ell : \mathcal{Y} \times \mathcal{Y} \to \mathcal{R}$ admits an ***Implicit Embedding*** if there exists

$$c : \mathcal{Y} \to \mathcal{H}, \quad V : \mathcal{H} \to \mathcal{H}$$

into a (separable) Hilbert space $\mathcal{H}$ such that

$$\ell(z,y) \;=\; \langle\, c(z),\, V\,c(y)\rangle_{\mathcal{H}}, \quad \forall\, y,z \in \mathcal{Y}.$$

which allows to embed the loss into a linear space.

***Example: Squared Loss as Implicit Embedding**

Let $\mathcal{Y} \in \mathcal{R}$ and $\ell(y,y') = (y-y')^2$. Define

$$c(y) = \begin{pmatrix} y^2 \\ \sqrt{2}\,y \\ 1 \end{pmatrix} \in \mathcal{R}^3, \qquad V = \begin{pmatrix} 0 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 0 \end{pmatrix}.$$

Then one verifies

$$\ell(y,y') = (y-y')^2 = c(y)^\top\, V\, c(y').$$

27.4 Kernel + Implicit Embedding

1. Linear closed-form. Given data $X \in^{n\times d}$, label-embeddings $Y \in^{n\times T}$, and ridge-penalty $\lambda > 0$:

$$g_n(x) \;=\; W_n\, x \quad W_n \;=\; Y^\top X\,(X^\top X + n\lambda I_d)^{-1}$$

Equivalently, one obtains the "dual" expansion

$$g_n(x) = \sum_{i=1}^n \alpha_i(x)\; c(y_i), \quad \alpha(x) \;=\; (XX^\top + n\lambda I_n)^{-1}\, X\, x.$$

2. Kernelization. Replace $x \mapsto \phi(x)$ (RKHS) with kernel $k(x,x') = \langle \phi(x), \phi(x')\rangle$. Define

$$K_{ij} = k(x_i, x_j), \quad v(x)_i = k(x_i, x).$$

Then

$$g_n(x) = \sum_{i=1}^n \alpha_i(x)\; c(y_i), \quad \alpha(x) = (K + n\lambda I_n)^{-1}\, v(x),$$

so **no infinite parameters need ever be stored**. All we have to compute are $\alpha_i(x)$.

3. Kernelized Ridge Surrogate. By the Representer Theorem, the solution of

$$W_n = \arg\min_{W\in\otimes} \frac{1}{n}\sum_{i=1}^n \|c(y_i) - W\,\phi(x_i)\|^2 + \lambda\|W\|_{HS}^2$$

has form $g_n(x) = W_n\phi(x) = \sum_i \alpha_i(x)c(y_i)$, which handles non-linear relations in x and avoids constructing $\phi(x)$. **Kernel methods let us learn a potentially infinite-dimensional implicit embedding by solving only $n\times n$ linear systems—no need to manipulate W_n explicitly.**

27.5 Loss Trick on Decoding Step

Writing the learned regressor as

$$g_n(x) = \sum_{i=1}^n \alpha_i(x)\, c(y_i),$$

the final predictor admits the "loss-trick" form

$$f_n(x) = \arg\min_{z\in\mathcal{Y}} \big\langle c(z),\, V\, g_n(x)\big\rangle = \arg\min_{z\in\mathcal{Y}} \sum_{i=1}^n \alpha_i(x)\,\ell\big(z, y_i\big).$$

27.6 Final Surrogate Framework

Algorithm 7 Surrogate + Implicit Embedding + Loss Trick

1. **Input:**

$$\{(x_i, y_i)\}_{i=1}^n, \quad \mathcal{Y} = \{1, \dots, T\}, \quad V \in \mathcal{R}^{T \times T}, \quad L : \mathcal{R}^T \times \mathcal{R}^T \to \mathcal{R} \quad \text{(convex surrogate loss)}, \quad \mathcal{G} \subset \{g : \mathcal{X} \to^T\}.$$

   $[V_{yz} = \ell(y, z) = e_y^\top V e_z]$

2. **Encoding labels y via $c(y)$:**

$$\forall i = 1, \dots, n : \quad c_i \; = \; e_{y_i} \quad \text{(one-hot encode each label)}$$

   To translate y into a fixed linear space.

3. **Surrogate learning a vector-valued regressor g_*:**

$$g_n \; = \; \arg\min_{g \in \mathcal{G}} \frac{1}{n} \sum_{i=1}^n \left\| g(x_i) - c_i \right\|^2 + \lambda \|g\|_{\mathcal{G}}^2 \quad \text{(regularized convex regression)}$$

   - $g_n(x)$ approximates the mean embedding of $c(y)$ given x: $E[c(y) \mid x]$ to capture how labels vary with inputs.
   - *Linear case:* set $g(x) = W\, x$, solve

$$W_n = Y^\top X \, (X^\top X + n\lambda I)^{-1}, \quad g_n(x) = W_n \, x.$$

   - *Kernelized case:* by representer theorem,

$$g_n(x) = \sum_{i=1}^n \alpha_i(x) \, c_i, \quad \alpha(x) = (K + n\lambda I)^{-1} v(x),$$

     where $K_{ij} = k(x_i, x_j)$, $v(x)_i = k(x_i, x)$.

4. **Decoding via Loss Trick (for new x):**

   (a) *Compute expansion:*

$$g_n(x) = \sum_{i=1}^n \alpha_i(x) \, c(y_i).$$

   (b) *Decode (Pick optimal z):*

$$\hat{y} = \arg\min_{z \in \mathcal{Y}} \sum_{i=1}^n \alpha_i(x) \, \ell(z, y_i).$$

   which avoids explicit knowledge of $(\mathcal{H}, c, V)$!

5. **Output:** Structured predictor

$$f_n(x) = \hat{y} = \arg\min_{z \in \mathcal{Y}} \sum_{i=1}^n \alpha_i(x) \, \ell(z, y_i).$$

27.6.1 Comparison Inequality Bound

Let g be any vector-valued function and its associated predictor $f = d \cdot g$, we have the bound:

$$\mathcal{E}(d \circ g) \; - \; \mathcal{E}(f^*) \; \leq \; 2\, Q \, \|V\|_{\text{op}} \, \sqrt{\mathcal{R}(g) \; - \; \mathcal{R}(g_*)},$$

- $\mathcal{E}(d \circ g) - \mathcal{E}(f^*)$: the *excess structured risk* of the predictor $f = d \circ g$ and Bayes optimal f^*, where

$$\mathcal{E}(f) = \int_{\mathcal{X} \times \mathcal{Y}} \ell\big(f(x), y\big) \, d\rho(x, y).$$

- $\mathcal{R}(g) - \mathcal{R}(g_*)$: the *surrogate least-squares risk gap*:

$$\mathcal{R}(g) = \int_{\mathcal{X}\times\mathcal{Y}} \big\|g(x) - c(y)\big\|_{\mathcal{H}}^2 \, d\rho(x,y), \quad g_*(x) = \mathbb{E}_{y|x}[\, c(y)\,].$$

which measures how well g approximates the ideal embedding $g_*(x)$.

- The constants Q and $\|V\|$ control how "large" the embedding and loss operator can be, scaling the bound:

$$Q = \sup_{y\in\mathcal{Y}} \|c(y)\|_{\mathcal{H}}, \qquad \|V\|_{\mathrm{op}} = \sup_{\|h\|\le 1} \|Vh\|_{\mathcal{H}}.$$

***Derivation of Comparison Inequality**

1. Rewrite excess risk via IE. Since $\ell(z,y) = \langle c(z), V\, c(y)\rangle_{\mathcal{H}}$,

$$\mathcal{E}(d\circ g) - \mathcal{E}(f^*) = \int_{\mathcal{X}\times\mathcal{Y}} \big\langle c\big(d(g(x))\big) - c\big(f^*(x)\big),\, V\, c(y)\big\rangle \, d\rho(x,y).$$

2. Integrate out y. Use $\int c(y)\, d\rho(y\,|\,x) = g_*(x)$:

$$= \int_{\mathcal{X}} \big\langle c\big(d(g(x))\big) - c\big(f^*(x)\big),\, V\, g_*(x)\big\rangle \, d\rho_{\mathcal{X}}(x).$$

3. Add and subtract mixed term. Insert $\langle c(d(g(x))), V\, g(x)\rangle$:

$$\begin{aligned}\mathcal{E}(d\circ g) - \mathcal{E}(f^*) = \int \Big[&\langle c(d(g(x))), V\, g(x)\rangle - \langle c(f^*(x)), V\, g_*(x)\rangle \\ &+ \ \langle c(d(g(x))), V\,(g_*(x) - g(x))\rangle\Big] \, d\rho_{\mathcal{X}}(x).\end{aligned}$$

Call the first bracketed difference A and the last term B.

4. Bound A via minima comparison. By decoding,

$$\langle c(d(g(x))), V\, g(x)\rangle = \min_{y\in\mathcal{Y}} \langle c(y), V\, g(x)\rangle,$$

and similarly for f^*. Thus

$$A \le \int \sup_{y\in\mathcal{Y}} \big|\langle c(y), V\,(g_*(x) - g(x))\rangle\big| \, d\rho_{\mathcal{X}}(x).$$

5. Bound B similarly. By convexity of absolute value,

$$B \ \le \ \int \sup_{y\in\mathcal{Y}} \big|\langle c(y), V\,(g_*(x) - g(x))\rangle\big| \, d\rho_{\mathcal{X}}(x).$$

6. Combine A and B.

$$\mathcal{E}(d\circ g) - \mathcal{E}(f^*) \ \le \ 2\int \sup_{y} \big|\langle c(y), V\,(g_*(x) - g(x))\rangle\big| \, d\rho_{\mathcal{X}}(x).$$

7. Apply Cauchy–Schwarz.

$$\big|\langle c(y), V\,\Delta g(x)\rangle\big| \ \le \ \|c(y)\|_{\mathcal{H}}\ \|V\|\ \|\Delta g(x)\|_{\mathcal{H}},$$

so taking $\sup_y$ introduces $Q\,\|V\|$.

8. Use Jensen's inequality.

$$\int \|\Delta g(x)\| \, d\rho_{\mathcal{X}}(x) \ \le \ \sqrt{\int \|\Delta g(x)\|^2 \, d\rho} = \sqrt{\mathcal{R}(g) - \mathcal{R}(g_*)}.$$

Combining yields

$$\mathcal{E}(d\circ g) - \mathcal{E}(f^*) \ \le \ 2\,Q\,\|V\|\ \sqrt{\mathcal{R}(g) - \mathcal{R}(g_*)}.$$